

课时5 计算机的运算方法

考点	重要程度	分值	常考题型
1.有符号数的编码方法	★★★	2 ~ 6	填空、判断、计算
2.定点数与浮点数	★★★★	2 ~ 8	选择、填空、计算
3.定点运算与浮点运算	★★★★	2 ~ 8	填空、计算



考点1 有符号数的编码方法 掌握



一、编码计算

- 1、原码：**最高位符号位0正1负，其余是二进制数的绝对值的大小
- 2、反码：**
 - ① 正数的表示与原码相同
 - ② 负数的符号位为1，数值位将原码的数值按位取反
- 3、补码：**
 - ① 正数的表示与原码、反码相同
 - ② 负数的符号位为1，数值位将原码的数值按位取反加一
- 4、移码：**移码和补码符号位相反，其他位相同

【题1】 写出下列数的原码、反码、补码和移码。

(1) $x = (10011)_2$

(2) $y = (-11010)_2$

解: $[x]_{\text{原}} = 010011$ $[y]_{\text{原}} = 111010$

$[x]_{\text{反}} = 010011$ $[y]_{\text{反}} = 100101$

$[x]_{\text{补}} = 010011$ $[y]_{\text{补}} = 100110$

$[x]_{\text{移}} = 110011$ $[y]_{\text{移}} = 000110$



扫码观看
视频讲解更清晰

二、0的编码表示

原码有两种表示0的方式: $[+0]_{\text{原}}=00000000, [-0]_{\text{原}}=10000000$

反码有两种表示0的方式: $[+0]_{\text{反}}=00000000, [-0]_{\text{反}}=11111111$

0在补码中的表示是唯一的: $[+0]_{\text{补}}=[-0]_{\text{补}}=00000000$

0在移码中的表示是唯一的: $[+0]_{\text{移}}=[-0]_{\text{移}}=10000000$

【题2】 针对8位二进制数，下列说法正确的是 (B)

A、-127的补码为10000000

B、-127的反码等于0的移码

C、+1的移码等于-127的反码

D、0的补码等于-1的反码

解：

$$(1) (-127)_{10} = (-111\ 1111)_2, [-111\ 1111]_{\text{原}} = 1111\ 1111, [-111\ 1111]_{\text{反}} = 1000\ 0000$$

$$[-111\ 1111]_{\text{补}} = 1000\ 0001$$

$$(2) [+1]_{\text{原}} = 0000\ 0001, [+1]_{\text{反}} = 0000\ 0001, [+1]_{\text{补}} = 0000\ 0001, [+1]_{\text{移码}} = 1000\ 0001$$

$$(3) [+0]_{\text{补}} = [-0]_{\text{补}} = 00000000, [+0]_{\text{移}} = [-0]_{\text{移}} = 10000000$$

$$(4) [-1]_{\text{原}} = 1000\ 0001, [-1]_{\text{反}} = 1111\ 1110$$

考点2 定点数与浮点数

理解



一、基本概念

1、数值数据的表示格式:

	定点格式	浮点格式
所有小数点位置	固定不变	不固定
数据范围	有限	很大
处理硬件	简单	复杂

一、基本概念

2、定点表示法：所有数据的小数点位置固定不变

(1) 理论上小数点位置可以任意，但实际上将数据表示有两种方法：

① 纯小数（定点小数）

② 纯整数（定点整数）

纯小数小数点固定在这

纯整数小数点固定在这

X_0 $X_1 X_2 X_3 \dots X_{n-1} X_n$

符号位 数值域

0: 正, 1: 负

一、基本概念

2、**定点表示法**：所有数据的小数点位置固定不变

(2) 定点表示法特点

- ① 定点数表示数的范围受字长限制，表示数的范围有限
- ② 定点表示的精度有限

【题3】一个定点数由 符号位 和 数值域 两部分组成。

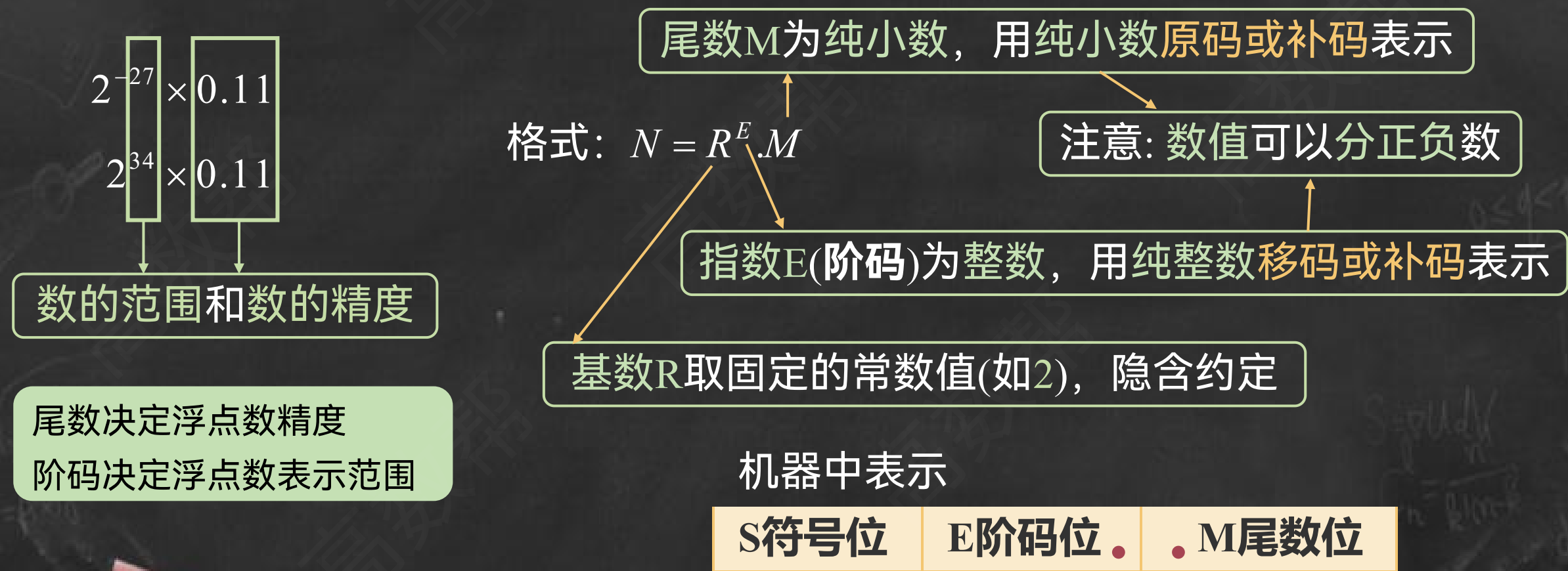


扫码观看
视频讲解更清晰

一、基本概念

3、**浮点表示法**：小数点的位置不固定，可以在一定范围内自由浮动

若有定点计算机无法直接表示的数值范围：则数的范围和数的精度分别存放



【题4】 长度相同但格式不同的2种浮点数，假设前者阶码长、尾数短，后者阶码短、尾数长，其他规定均相同，则他们可表示的数的范围和精度为（ **B** ）

A、两者可表示的数的范围和精度相同

B、前者可表示的数的范围大但精度低

C、后者可表示的数的范围大且精度高

D、前者可表示的数的范围大且精度高

二、规格化浮点数

1、**浮点数规格化**：能够使得计算机进行浮点运算时有固定统一的标准格式

2、**规格化浮点数对尾数限制范围**：

(1) **正数0**：规格化表示的尾数形式为： $0.1xx\dots x$

(2) **负数1**：

1) 补码表示的负数，规格化表示的尾数形式为： $1.0xx\dots x$ （即真值为 $-0.1xx\dots x$ ）

2) 原码表示的负数，规格化表示的尾数形式为： $1.1xx\dots x$ （即真值为 $-0.1xx\dots x$ ）

【题5】若浮点数用补码表示，则判断运算结果为规格化数的方法是（C）。

A、阶符与数符相同，则为规格化数

B、小数点后第一位为1，则为规格化数

C、数符与小数点后第1位数字相异，则为规格化数

D、数符与小数点后第1位数字相同，则为规格化数



扫码观看
视频讲解更清晰

二、规格化浮点数

3、浮点数规格化处理:

(1)右规: 尾数每右移一位, 阶码加1

(2)左规: 尾数每左移一位, 阶码减1

(3)思想: $(1.75)_{10} = (1.75)_{10} \times 10^0$

右规一位: $(1.75)_{10} = (0.175)_{10} \times 10^1$

左规一位: $(1.75)_{10} = (17.5)_{10} \times 10^{-1}$

$$(1.75)_{10} = (1.11)_2 \times 2^0 = (0.111)_2 \times 2^1 = (0.0111)_2 \times 2^2$$

$$(-1.75)_{10} = (-1.11)_2 \times 2^0 = (-0.111)_2 \times 2^1 = (-0.0111)_2 \times 2^2$$

其中 $(0.111)_2 \times 2^1$ 和 $(-0.111)_2 \times 2^1$ 为浮点数规格化表示

【题6】 假设浮点数据为16位，1位符号位，阶码5位，用补码表示；尾数10位，用补码表示。写出二进制数 $(-1011.10111)_2$ 的规格化浮点数的机器表示。

(题型：已知真值求规格化浮点数)

解：

(1) 对浮点数进行规格化表示： $(-1011.10111)_2 = (-0.101110111)_2 \times 2^4$

(2) 符号位为负数1

(3) 指数真值为 $(+4)_{10} = (+100)_2$ ，原码为0100，由于阶码5位，原码补充0得00100
对应阶码位为00100

(4) 尾数真值为 -0.101110111 ，原码为1.101110111

由于尾数10位，原码补充0得1.1011101110，对应补码为1.0100010010

则尾数位为0100010010

(5) 综上所述，规格化浮点数为1 00100 0100010010

三、IEEE标准浮点数

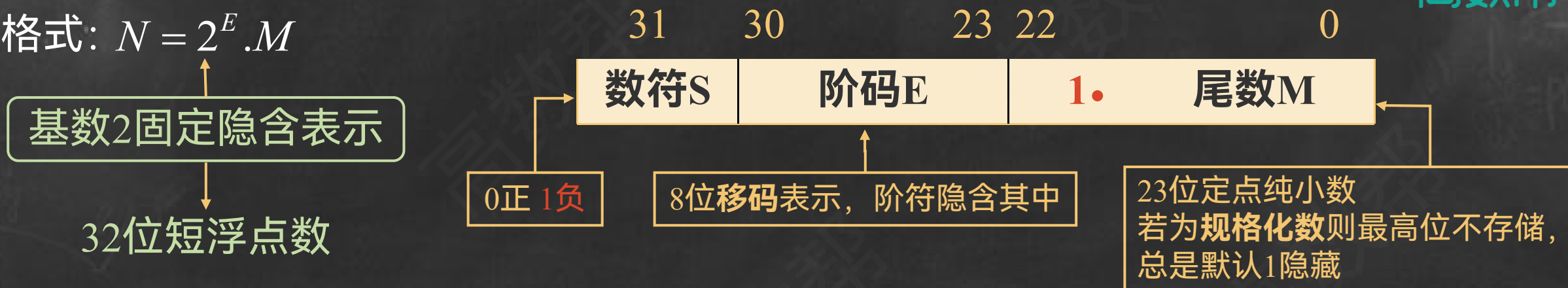
1、IEEE754标准

- (1) 规则规定了单精度(32)和双精度(64)的基本格式.
- (2) 规则中, 尾数M用原码, 指数E用移码(便于对阶和比较)
- (3) 基数 $R=2$, 基数固定, 采用隐含方式来表示它。

【题7】 移码表示法主要用于表示浮点数的阶码, 以利于比较两个指数的大小和进行对阶操作。

(4) 32位的IEEE754标准浮点数:

格式: $N = 2^E \cdot M$



① S数的符号位, 1位, 在最高位, “0”表示正数, “1”表示负数。

② E是阶码, 8位, 采用移码表示。移码比较大小方便。

注意: 采用这种方式时, 将浮点数的指数真值 e 变成阶码 E 时, 应将指数 e 加上一个固定的偏移值127, 即 $E = e + 127$, 使得表示范围由原来的 $-126 \sim +127$, 变为 $1 \sim 254$

③ M是尾数, 23位, 在低位部分, 采用纯小数表示。

(尾数域最左位(最高有效位)总是1, 故这一位经常不予存储, 而认为隐藏在小数点的左边)

④ 一个规格化的32位浮点数 x 的真值表示为 $x = (-1)^S \times (1.M) \times 2^{E-127}$ (指数 $e = E - 127$)

【题8】 将数值-0.5按IEEE754单精度格式存储。(题型：已知真值求IEEE754标准浮点数)

解：

首先将十进制转换成二进制： $(-0.5)_{10}=(-0.1)_2$

然后移动小数点，使其在第1.M位之间

$$-0.1 = -1.0 \times 2^{-1}$$

(1)符号位S 1位： $S = 1$

(2)阶码E 8位： $E = \text{指数真值}e + 127 = -1 + (0111\ 1111)_2 = (0111\ 1110)_2$

(3)尾数M 23位： $M = 0000\ 0000\ 0000\ 0000\ 0000\ 000$

(4)则浮点表示为： $1\ 0111\ 1110\ 0000\ 0000\ 0000\ 0000\ 0000\ 000$

【题9】若采用IEEE754浮点单精度格式，试求出32位浮点代码 $(CC678000)_{16}$ 的真值。

(题型：已知IEEE754标准浮点数求真值)

解：

首先将十六进制数转换为二进制数：

1, 100 1100 0, 110 0111 1000 0000 0000 0000

(1) 符号1位： $S = 1$ ，负数

(2) 阶码8位： $E = 100\ 1100\ 0$

$$\begin{aligned}\text{真值 } e &= \text{阶码 } E - 127 = (10011000)_2 - (127)_{10} \\ &= 152 - 127 = 25\end{aligned}$$

尾数23位： $M = 110\ 0111\ 1000\ 0000\ 0000\ 0000$

$$\begin{aligned}\text{包括隐藏位1的尾数真值 } 1.M &= 1 + 0.11001111 = (1.11001111)_2 \\ &= (1.808593785)_{10}\end{aligned}$$

$$\text{浮点数真值} = (-1)^S \times (1.M) \times 2^{E-127} = -1.808593785 \times 2^{25}$$



扫码观看
视频讲解更清晰

考点3 定点运算与浮点运算

掌握



二、定点数的补码加减法

(1) 补码加法: $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$

(2) 补码减法: $[X-Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$

其中: $[Y]_{\text{补}}$ 含符号位, 取反加一求得 $[-Y]_{\text{补}}$

【题10-1】 $x = +1001$, $y = +0101$,
请用补码加法求 $x+y = ?$

$$\begin{array}{r} \text{解: } [x]_{\text{补}} = 01001, [y]_{\text{补}} = 00101 \\ \begin{array}{r} [x]_{\text{补}} \quad 01001 \\ + [y]_{\text{补}} \quad 00101 \\ \hline [x+y]_{\text{补}} \quad 01110 \end{array} \end{array}$$

则 $[x+y]_{\text{原}} = 01110$

$\therefore x+y = +1110$

【题10-2】 $x = -1001$, $y = +0101$,
请用补码加法求 $x+y = ?$

$$\begin{array}{r} \text{解: } [x]_{\text{补}} = 10111, [y]_{\text{补}} = 00101 \\ \begin{array}{r} [x]_{\text{补}} \quad 10111 \\ + [y]_{\text{补}} \quad 00101 \\ \hline [x+y]_{\text{补}} \quad 11100 \end{array} \end{array}$$

则 $[x+y]_{\text{原}} = 10100$

$\therefore x+y = -0100$

**注意：已知补码转原码，规则也是
正数不变**

负数除符号位取反加一

【题11】 $x = +1101$, $y = +0110$, 请用补码减法求 $x - y = ?$

解: $[x]_{\text{补}} = 01101$
 $[y]_{\text{补}} = 00110$, $[-y]_{\text{补}} = 11010$

$$\begin{array}{r} [x]_{\text{补}} \quad 01101 \\ + [-y]_{\text{补}} \quad 11010 \\ \hline \end{array}$$

$$[x-y]_{\text{补}} \quad \boxed{1}00111$$

↓
自然丢失

则 $[x-y]_{\text{原}} = 00111$

$\therefore x - y = +0111$

三、定点数的溢出检测：双符号位法（变形补码）

变形补码：任何正数，两符号位都是0；任何负数，两符号位都是1

最后结果：两符号位 $Sf1$ $Sf2$

$Sf1$ $Sf2$

0 0 正确（正数）

0 1 正溢

1 0 负溢

1 1 正确（负数）



扫码观看
视频讲解更清晰

【题12】 $x = +1100$, $y = +1000$, 用变形补码求 $x+y$, 并说明结果是否溢出, 如果溢出, 请说明产生正溢还是负溢, 如果不溢出, 请求出 $x+y$ 。

解: $[x]_{\text{补}} = 001100$, $[y]_{\text{补}} = 001000$

$$\begin{array}{r} [x]_{\text{补}} \quad 001100 \\ + [y]_{\text{补}} \quad 001000 \\ \hline \end{array}$$

$$[x+y]_{\text{补}} \quad 010100$$

结果为正, 同时产生正溢。

【题13】 $x = +1101$, $y = +0110$, 用变形补码求 $x-y$, 并说明结果是否溢出, 如果溢出, 请说明产生正溢还是负溢, 如果不溢出, 请求出 $x-y$ 。

解: $[x]_{\text{补}} = 001101$

$[y]_{\text{补}} = 000110$, $[-y]_{\text{补}} = 111010$

$$\begin{array}{r} [x]_{\text{补}} \quad 001101 \\ + \quad [-y]_{\text{补}} \quad 111010 \\ \hline \end{array}$$

$$[x-y]_{\text{补}} \quad \boxed{1}000111$$

自然丢失

$$[x-y]_{\text{原}} = 000111$$

$$\therefore x-y = +0111$$

计算结果不溢出



四、定点数的乘法

【题14】 已知带符号的 $x = +15$, $y = -13$, 要求使用二进制定点数乘法求出乘积 $x \cdot y = ?$

解: $[x]_{\text{原}} = 01111$, $[y]_{\text{原}} = 11101$, $|x| = 1111$, $|y| = 1101$

符号位运算: $0 \oplus 1 = 1$

$$\begin{array}{r} \\ \\ \times \\ \hline \\ \\ \\ \\ \\ + \\ \hline \end{array}$$

输出11000011, $[x \times y]_{\text{原}} = 111000011$

换算成二进制数真值是 $x \cdot y = (-11000011)_2 = (-195)_{10}$

五、浮点数的加减法

1、浮点运算步骤如下:

- (1) 0 操作数检查,有一个操作数为0则没必要进行后续操作;
- (2) 比较阶码大小并对阶:阶码相同则小数点位置对齐, 若不同则**小阶向大阶对齐**;
若 $\Delta E = 0$, 表示两数阶码相等,即 $E_x = E_y$;
若 $\Delta E > 0$, 表示 $E_x > E_y$;
若 $\Delta E < 0$, 表示 $E_x < E_y$ 。
当 $E_x \neq E_y$ 时, 将小阶的操作数的尾数 M 向右移动 ΔE 位,使 E_x 和 E_y 相等。
- (3) **尾数进行加或减运算, 方法与定点加减法运算完全一样;**
- (4) 结果规格化并进行舍入处理
 - ① 左规: 尾数左移才能满足规格化条件
 - ② 右规: 尾数右移才能满足规格化条件
 - ③ 就近舍入(0舍1入): 类似“四舍五入”,丢弃的最高位为1, 进1

【题15】 设 $x = 2^{010} \times 0.11011011$, $y = -2^{100} \times 0.10101100$, 要求用二进制带符号数补码求 $x+y$, 尾数保留8位。

解:

1、已知 $x=2^2 \times 0.11011011$, $y=2^4 \times (-0.10101100)$, 无操作数0

2、对阶: $x=2^4 \times 0.00110110(11)$

3、尾数相加: $[Mx]_{\text{补}}=0.00110110(11), [My]_{\text{补}}=1.01010100$

$$\begin{array}{r} 00.00110110(11) \\ + 11.01010100 \\ \hline \end{array}$$

$$11.10001010(11)$$

$[Mx+y]_{\text{补}}=[Mx]_{\text{补}}+[My]_{\text{补}}=11.10001010(11), Ex+y=4$, 尾数不溢出

$$Mx+y=-0.0111010101$$

(注: 补码变真值方法也是正数相同, 负数除符号位外取反加一)

$$\text{即 } x+y=-0.0111010101 \times 2^4$$

4、结果规格化, 并检查是否溢出:

$$x+y=-0.01110101(01) \times 2^4=-0.111010101 \times 2^3, -126 \leq 3 \leq 127, \text{ 不溢出。}$$

5、舍入: 采用0舍1入法处理, 最终结果: $x+y = 2^{011} \times (-0.11101011)$

小结

计算机的运算方法

有符号数的编码

编码计算

- 原码 最高位符号位0正1负，其余是二进制数的绝对值的大小
- 反码
 - 正数的表示与原码相同
 - 负数的符号位为1，数值位将原码的数值按位取反
- 补码
 - 正数的表示与原码、反码相同
 - 负数的符号位为1，数值位将原码的数值按位取反加一
- 移码 移码和补码符号位相反，其他位相同

0的编码

- 原码
 - $[+0]_{\text{原}} = 00000000$
 - $[-0]_{\text{原}} = 10000000$
- 反码
 - $[+0]_{\text{反}} = 00000000$
 - $[-0]_{\text{反}} = 11111111$
- 补码 $[+0]_{\text{补}} = [-0]_{\text{补}} = 00000000$
- 移码 $[+0]_{\text{移}} = [-0]_{\text{移}} = 10000000$

小结

计算机的运算方法

定点数与浮点数

基本概念

定点表示法

所有数据的小数点位置固定不变

纯小数（定点小数）

纯整数（定点整数）

特点

定点数表示数的范围受字长限制，表示数的范围有限

定点表示的精度有限

浮点表示法

小数点的位置不固定，可以在一定范围内自由浮动

特点

若有定点计算机无法直接表示的数值范围：则数的范围和数的精度分别存放

尾数决定浮点数精度

阶码决定浮点数表示范围

规格化浮点数

能够使得计算机进行浮点运算时有固定统一的标准格式

规格化浮点数对尾数限制范围

正数0：规格化表示的尾数形式为： $0.1xx\dots x$

负数1

补码表示的负数，规格化表示的尾数形式为： $1.0xx\dots x$ （即真值为 $-0.1xx\dots x$ ）

原码表示的负数，规格化表示的尾数形式为： $1.1xx\dots x$ （即真值为 $-0.1xx\dots x$ ）

IEEE754标准浮点数

浮点数规格化处理

右规：尾数每右移一位，阶码加1

左规：尾数每左移一位，阶码减1

小结

计算机的运算方法

定点数与浮点数

基本概念

- 规则规定了单精度(32)和双精度(64)的基本格式
- 规则中,尾数 M 用原码, 指数 E 用移码(便于对阶和比较)
- 基数 $R=2$, 基数固定, 采用隐含方式来表示它
- 32位的IEEE754标准浮点数

规格化浮点数

S 数的符号位, 1位, 在最高位, “0”表示正数, “1”表示负数

M 是尾数, 23位, 在低位部分, 采用纯小数表示。(尾数域最左位(最高有效位)总是1, 故这一位经常不予存储, 而认为隐藏在小数点的左边)

E 是阶码, 8位, 采用移码表示。移码比较大小方便 (阶码 E =真值 $e+127$)

32位浮点数 x 的真值表示为 $x=(-1)^S \times (1.M) \times 2^{E-127}$ (指数 $e=E-127$)

IEEE754标准 浮点数

小结

计算机的运算方法

定点运算与浮点运算

定点运算

移位运算

逻辑移位：左移：低位补0 右移：高位补0

循环移位：两端循环

算数移位

正数的表示与原码、反码相同

负数

原码负数：符号位不变（为1），空位补0

补码负数：左移补0，右移补1

反码负数：左移补1，右移补1

补码加减法

补码加法： $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$

补码减法： $[X-Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$

溢出判断：双符号位法（变形补码）

0 0 正确（正数）

0 1 正溢

1 0 负溢

1 1 正确（负数）

乘法运算

浮点加减法运算



扫码观看
视频讲解更清晰

小结

计算机的运算方法

定点运算与浮点运算

定点运算

0 操作数检查,有一个操作数为0则没必要进行后续操作

比较阶码大小并对阶:阶码相同则小数点位置对齐, 若不同则小阶向大阶对齐

尾数进行加或减运算, 方法与定点加减法运算完全一样

结果规格化并进行舍入处理

左规: 尾数左移才能满足规格化条件

右规: 尾数右移才能满足规格化条件

就近舍入(0舍1入): 类似“四舍五入”, 丢弃的最高位为1, 进1

浮点加减法运算

课时6 指令系统

考 点	重要程度	分值	常考题型
1. 机器指令	★★★	2 ~ 4	选择、判断、简答
2. 操作数类型和操作类型	★★★	2 ~ 4	选择、填空
3. 寻址方式	★★★★	2 ~ 8	选择、填空、计算



考点1 机器指令

理解



一、指令格式

操作码字段

地址码字段

1、操作码

概念：表示该指令应进行什么性质的操作，如进行加法、减法、乘法、除法、取数、存数等等，不同的指令用操作码字段的不同编码来表示，组成操作码字段的位数一般取决于计算机指令系统的规模。

【题1】某指令系统有200条指令，对操作码采用固定长度二进制编码时，最少需要用(**B**)位。

A、4

B、8

C、16

D、32

一、指令格式

1、操作码

操作码位数:

(1) 固定长度

优点: 指令规整, 译码简单

缺点: 信息的冗余极大,
使程序的总长度增加。

(2) 可变长度 (扩展操作码技术):

操作码的位数随地址数的减少而增加

4位	4位	4位	4位
OP	A1	A2	A3
0000	X	Y	Z
⋮	⋮	⋮	⋮
1110	X	Y	Z
1111	0000	Y	Z
⋮	⋮	⋮	⋮
1111	1110	Y	Z
1111	1111	0000	Z
⋮	⋮	⋮	⋮
1111	1111	1110	Z
1111	1111	1111	0000
⋮	⋮	⋮	⋮
1111	1111	1111	1111

三地址指令
15条操作码

二地址指令
15条操作码

一地址指令
15条操作码

零地址指令
16条操作码

扩展操作码实例

2、地址码

概念：可表示操作数，操作数的地址，运算结果的地址，或下一条指令的地址。

如寄存器编号、外围设备端口地址、内存单元地址、操作数数值本身。

根据一条指令中有几个操作数地址，可将该指令称为几操作数指令或几地址指令：

(1) 零地址指令

① 其格式为：

OP

操作码OP

② 功能描述：

不需要操作数的指令，如停机、空操作、清除等控制类指令

2、地址码

(2) 单地址指令

① 指令格式为：



操作码OP 第一操作数地址A1

② 功能描述：

- 1) 只有目的操作数的单操作数指令，如“+1”、“-1”、“求反” $OP(A) \rightarrow A$ ；
- 2) 隐含另一个操作数地址的双操作数指令 $(AC)OP(A) \rightarrow AC$ ；

【题2】下列关于一地址指令的说法正确的是 (D)

- A、只有一个操作数
- B、一定有两个操作数，其中一个是隐含的，完成功能 $(A)OP(ACC)$
- C、如果有两个操作数，则两个操作数相同，完成功能 $(A)OP(A)$
- D、可能有两个操作数，也可能只有一个操作数

2、地址码

(3) 二地址指令

① 其格式如下：

OP	A1	A2
----	----	----

操作码OP 目的操作数地址A1 源操作数地址A2

② 功能描述： $(A1) \text{ OP } (A2) \rightarrow A1$ （A1为目的操作数地址）

或 $(A1) \text{ OP } (A2) \rightarrow A2$ （A2为目的操作数地址）

③ 注意：指令执行之后，目的操作数地址原存的内容已经被新的运算结果替换了。

④ 二地址地址根据操作数的物理位置分为：

1)SS型 存储器-存储器类型：只访问内存的指令格式（速度最慢）

2)RS型 寄存器-存储器类型：既访问内存又访问寄存器的指令格式

3)RR型 寄存器-寄存器类型：只访问寄存器的指令格式（速度最快）

【题3】寻址方式按操作数的物理位置不同，多使用RR型和RS型，前者比后者执行速度快。

2、地址码

(4) 三地址指令

① 指令格式如下:

OP	A1	A2	A3
----	----	----	----

操作码OP

第一源操作数地址A1 第二源操作数地址A2

目的操作数地址A3

② 功能描述:

$(A1) \text{ op } (A2) \rightarrow A3$



扫码观看
视频讲解更清晰

2、地址码

(5) 四地址指令

① 指令格式如下:

OP	A1	A2	A3	A4
----	----	----	----	----

操作码 op

第一源地址 $A1$

第二源地址 $A2$

目的地址 $A3$

下指令地址

② 功能描述:

$(A1) \text{ op } (A2) \rightarrow A3$ $A4$ 给出下一条指令地址

【题4】四地址指令 $OPA1A2A3A4$ 的功能为 $(A1)OP(A2) \rightarrow A3$ ，且 $A4$ 给出下一条指令地址，假设 $A1, A2, A3, A4$ 都为主存储器地址，则完成上述指令需要访存（**C**）次。

A、2

B、3

C、4

D、5

二、指令字长

1、基本概念

(1) **指令字长**：一个指令字包含二进制代码的位数。

指令字长决定于 {
操作码的长度
操作数地址的长度
操作数地址的个数

(2) **(回忆)机器字长**：计算机能一次性处理的二进制数据的位数。

(3) **单字长指令**：指令字长等于机器字长的指令。

(4) **半字长指令**：指令字长等于半个机器字长的指令。

(5) **双字长指令**：指令字长等于两个机器字长的指令。

二、指令字长

2、多字长指令的优缺点

优点：提供足够的地址位来解决访问内存任何单元的寻址问题；

缺点：必须两次或多次访问内存以取出一整条指令，降低了CPU的运算速度，又占用了更多的存储空间。

3、等长指令与非等长指令

等长指令：各种指令字长度是相等的，指令字结构简单，且指令字长度是不变的。

非等长指令：各种指令字长度随指令功能而异，结构灵活，能充分利用指令长度，但指令的控制较复杂。

考点2 操作数类型与操作类型

理解



一、操作数类型

- 1、地址数据：无符号整数
- 2、数值数据：定点数、浮点数、压缩十进制数
- 3、字符数据：文本数据或字符串，目前广泛使用ASCII码
- 4、逻辑数据：逻辑运算

二、操作类型

- 1、**数据传送指令**：将数据从一个地方传送到另一个地方。
- 2、**算术运算指令**：
 - ① 二进制定点数、浮点数的加、减、乘、除
 - ② 十进制数加、减
 - ③ 求补运算
- 3、**逻辑运算指令**：
 - ① 与、或、非、异或
 - ② 移位

二、操作类型



高数帮

4、程序控制指令：程序控制类指令用于控制程序的执行方向，指出下一条指令地址，并使程序具有测试、分析与判断的能力。

(1) 转移指令

- 1) 无条件转移：不受任何条件约束转移。
- 2) 条件转移：按状态寄存器标志转移。

(2) 循环控制指令

(3) 子程序调用和返回指令

(4) 程序自中断指令

保存的标志包括：

进位标志(C)

结果为零标志(Z)

结果为负标志(S)

结果溢出标志(O)

结果奇偶标志(P)

二、操作类型



5、串处理类指令：对连续存储单元的内容进行处理串传送、串比较、串查找等

6、输入输出类指令：实现CPU与外设的信息传输

①外设向CPU输入数据指令

②CPU向外设输出数据指令

③启动外设指令

④检测外设状态指令

7、系统控制类指令

特权指令、标志位设置或复位指令、停机指令

特权指令用途：任务的创建和切换，检测用户的访问权限，改变系统的工作方式。

【题5】 程序控制类指令的功能是（D）

- A、进行算数运算和逻辑运算
- B、进行主存和CPU之间的数据传送
- C、进行CPU和I/O设备之间的数据传送
- D、改变程序执行的顺序

【题6】 运算型指令的寻址和转移性指令的寻址不同点在于（A）

- A、前者取操作数，后者决定程序转移地址
- B、后者决定操作数，前者决定程序转移地址
- C、前者是短指令，后者是长指令
- D、前者是长指令，后者是短指令

考点3 寻址方式 掌握

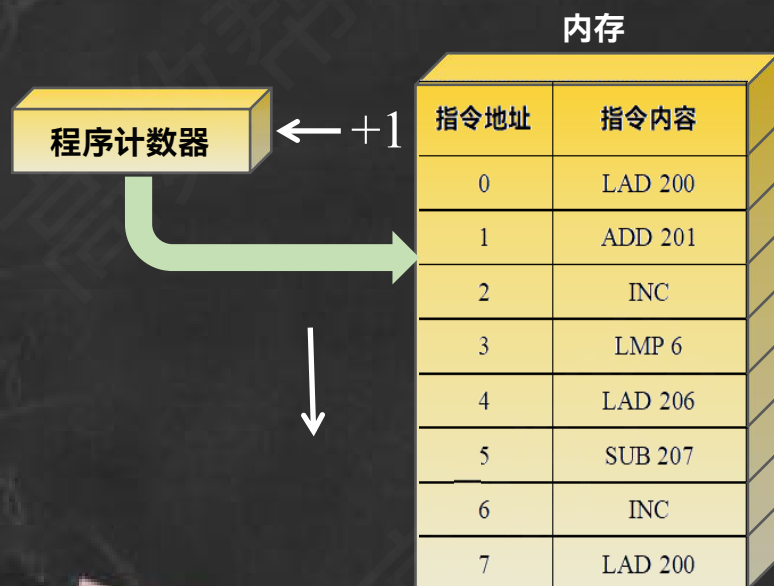
一、指令寻址：寻找要处理的指令

1、顺序方式

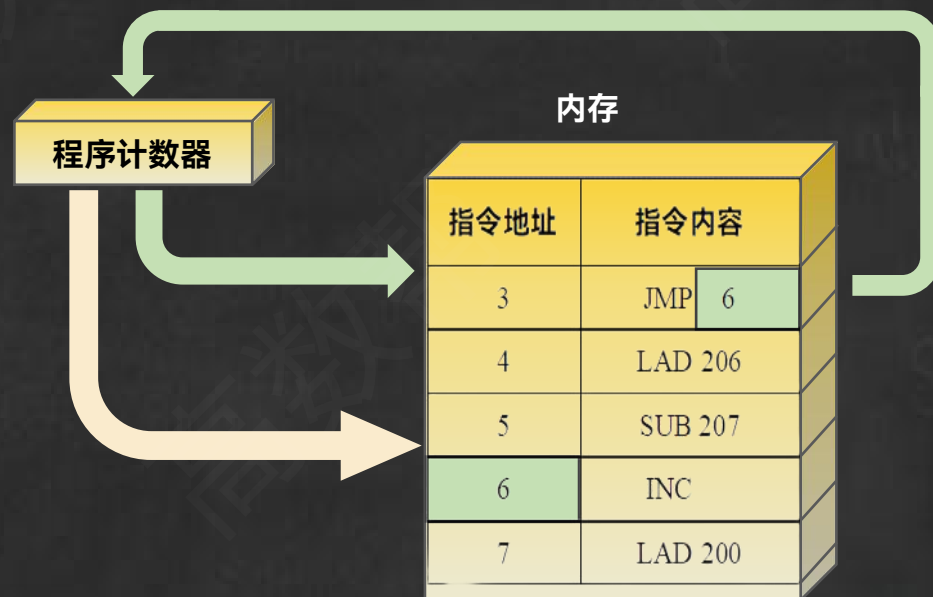
PC给出下条指令地址

2、跳跃方式

本条指令给出下条指令地址，PC的内容需相应改变



(a) 指令的顺序寻址方式



(b) 指令的跳跃寻址方式

【题7】指令的寻址方式有顺序和跳跃两种方式。采用跳跃寻址方式，可以实现（D）

A、堆栈寻址

B、程序的条件转移

C、程序的无条件转移

D、程序的条件转移或无条件转移



扫码观看
视频讲解更清晰

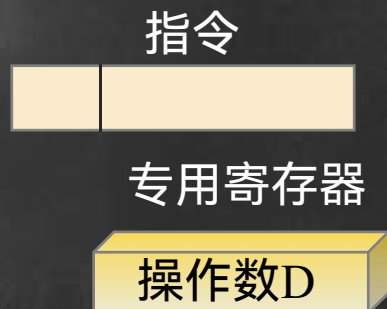
二、数据寻址：寻找操作数的地址

题型1 操作数来源：1 指令，2 寄存器，3 内存单元，4 堆栈

题型2 有效地址EA：操作数的实际地址

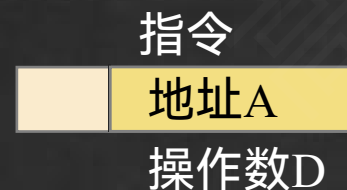
题型3 操作数内容

题型4 寻址方式分析



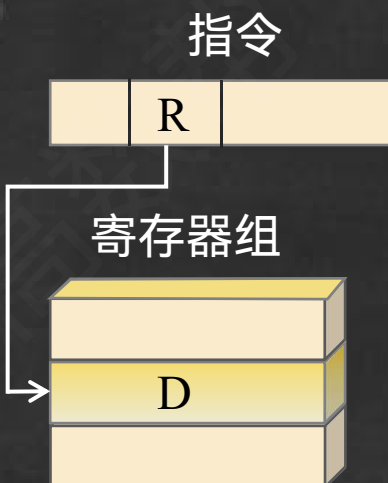
(1) 隐含寻址

操作数在专用寄存器
隐含地址



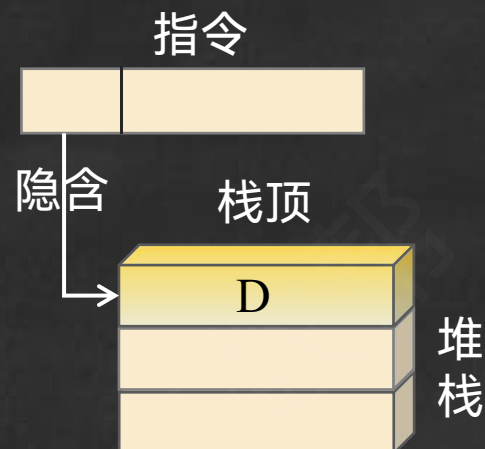
(2) 立即寻址

操作数在指令（地址码字段）
操作数D=形式地址A



(3) 寄存器寻址

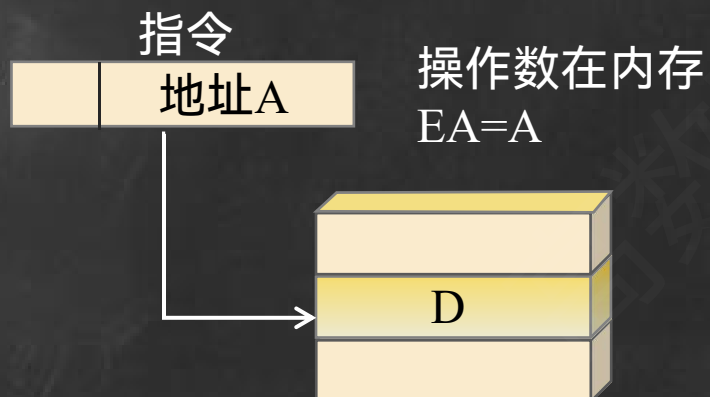
操作数在通用寄存器
EA=R



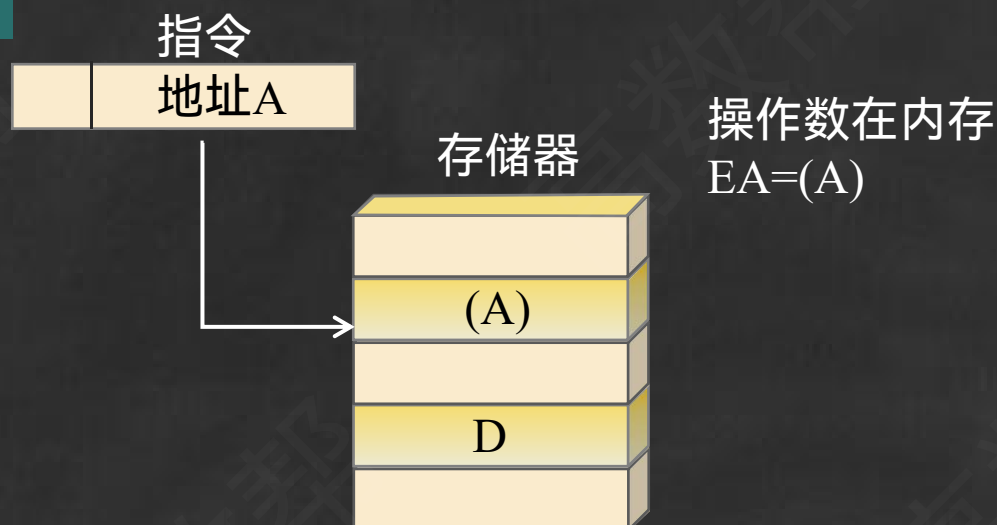
(4) 堆栈寻址

操作数在堆栈
EA=栈顶

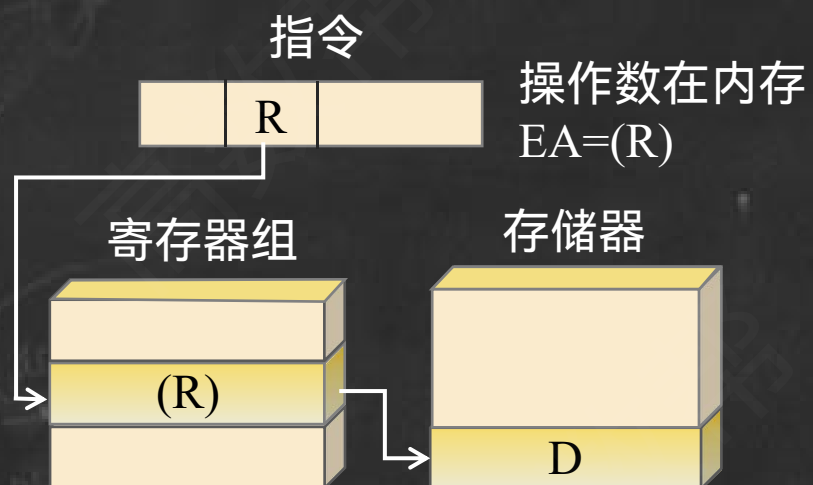
二、数据寻址：寻找操作数的地址



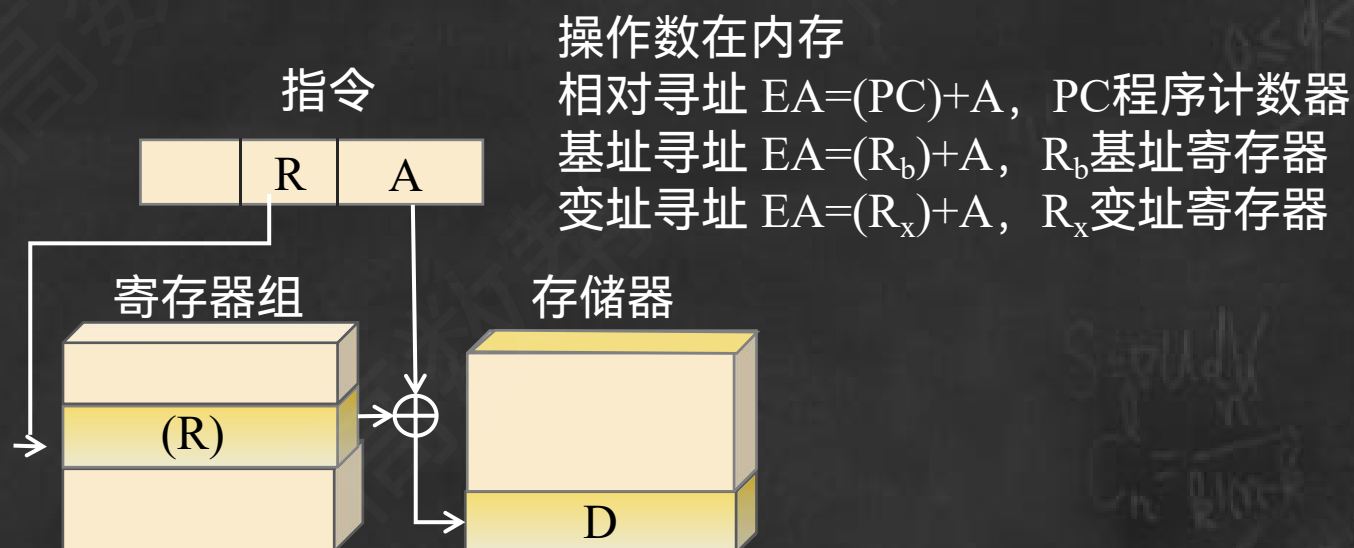
(5)直接寻址



(6)间接寻址



(7)寄存器间接寻址



(8)偏移寻址 $EA=(R)+A$

【题8】 寄存器间接寻址方式中，操作数处在（**B**）（操作数来源题型）

A、通用寄存器

B、主存单元

C、程序计数器

D、堆栈



扫码观看
视频讲解更清晰

【题9】变址寻址方式中，操作数的有效地址等于（C）（求有效地址题型）

A、基址寄存器内容加上形式地址（位移量）

B、堆栈指示器内容加上形式地址

C、变址寄存器内容加上形式地址

D、程序计数器内容加上形式地址

二、数据寻址：寻找操作数的地址

1、隐含寻址

某个操作数（或操作数地址）隐含在某个通用寄存器（或指定的存储单元）中。

例：如某些运算，隐含了累加器AC作为源和目的寄存器。如8086汇编中的STC指令，设置标志寄存器的C为1。

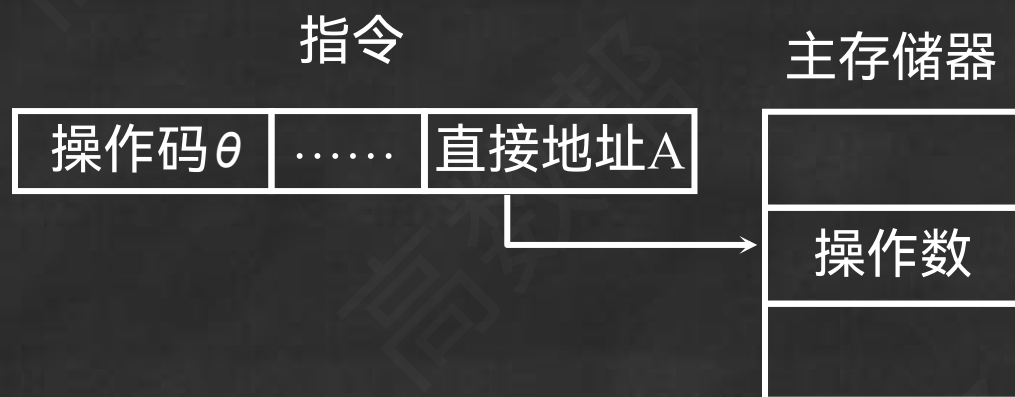
2、立即寻址

指令的地址码部分是操作数本身。如MOV AX, 8

二、数据寻址：寻找操作数的地址

3、直接寻址

指令的地址码部分给出操作数的有效地址EA，即 $EA=A$



二、数据寻址：寻找操作数的地址

4、间接寻址

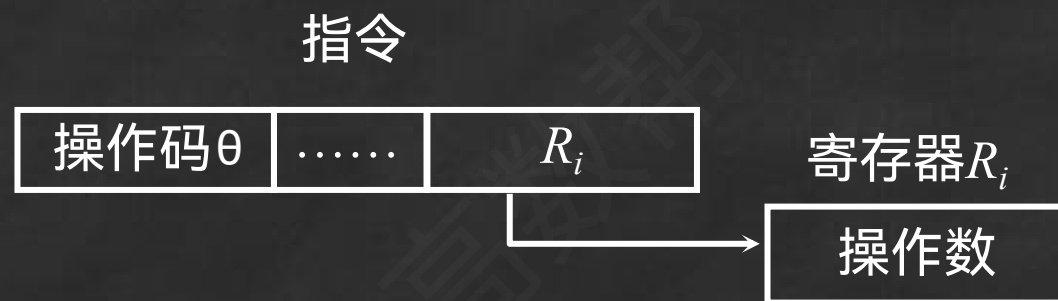
指令的地址码部分给出存放操作数地址的存储单元的地址（简称：地址的地址），
即 $EA = (A)$



二、数据寻址：寻找操作数的地址

5、寄存器寻址

指令的地址码字段直接给出寄存器编号，指令的操作数存放在相应的寄存器中，如MOV AX, BX，即 $EA=R_i$ 。



优点：

- ①由于寄存器在CPU的内部，指令在执行时从寄存器中取操作数比访问主存要快得多；
- ②由于寄存器的数量较少，因此寄存器编号所占位数也较少，从而可以有效减少指令的地址码字段的长度。

二、数据寻址：寻找操作数的地址

6、寄存器间接寻址

在指令的地址码部分给出该通用寄存器的编号，而该通用寄存器中存放着操作数在主存储器中的地址，即 $EA=(R_i)$ 。



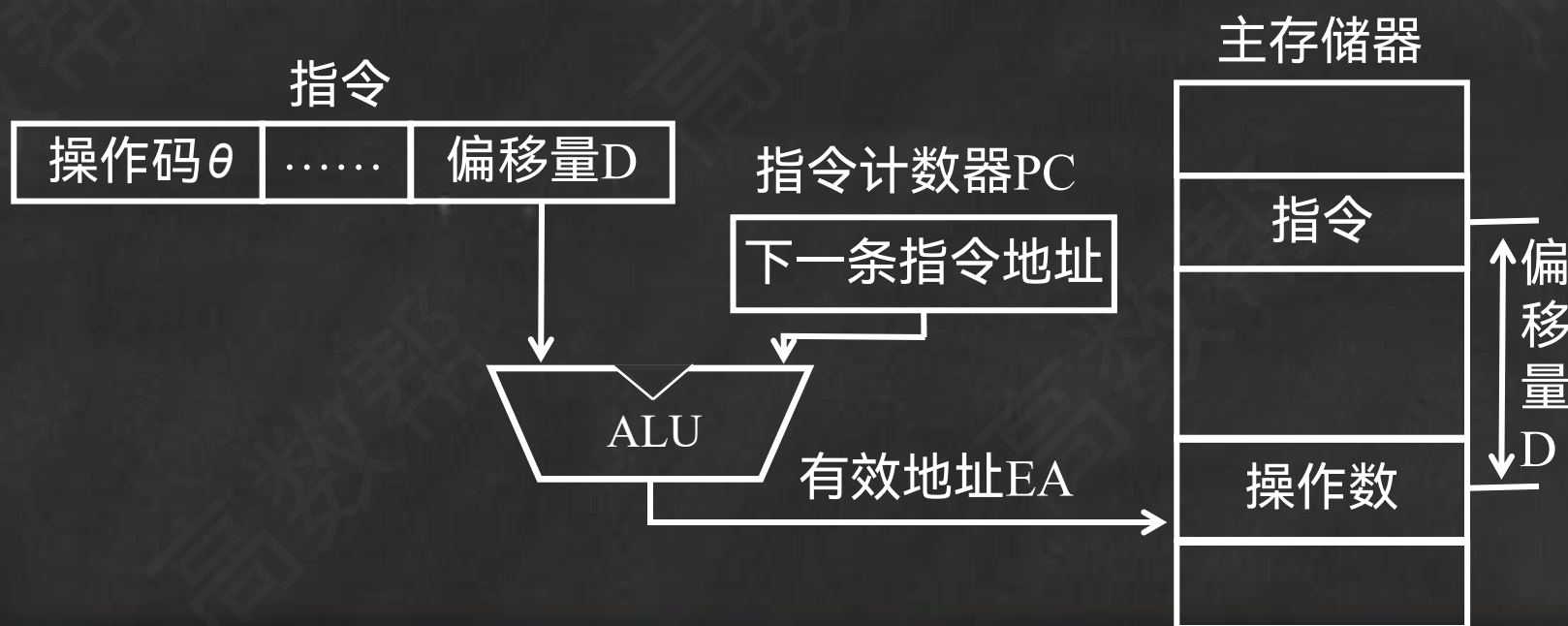
这种寻址方式的指令较短，并且在取指后只需一次访存便可得到操作数，因此指令执行速度较前述的间接寻址方式要快，也是目前在计算机中使用较为广泛的一种寻址方式。

二、数据寻址：寻找操作数的地址

7、偏移寻址

指定某寄存器R放置基准地址，指令的地址码部分给出一个相对位移量D，该寄存器的内容与形式地址相加为操作数的有效地址，即 $EA = (R) + D$ 。根据存放基准地址的寄存器不同，分为相对寻址、基址寻址、变址寻址。

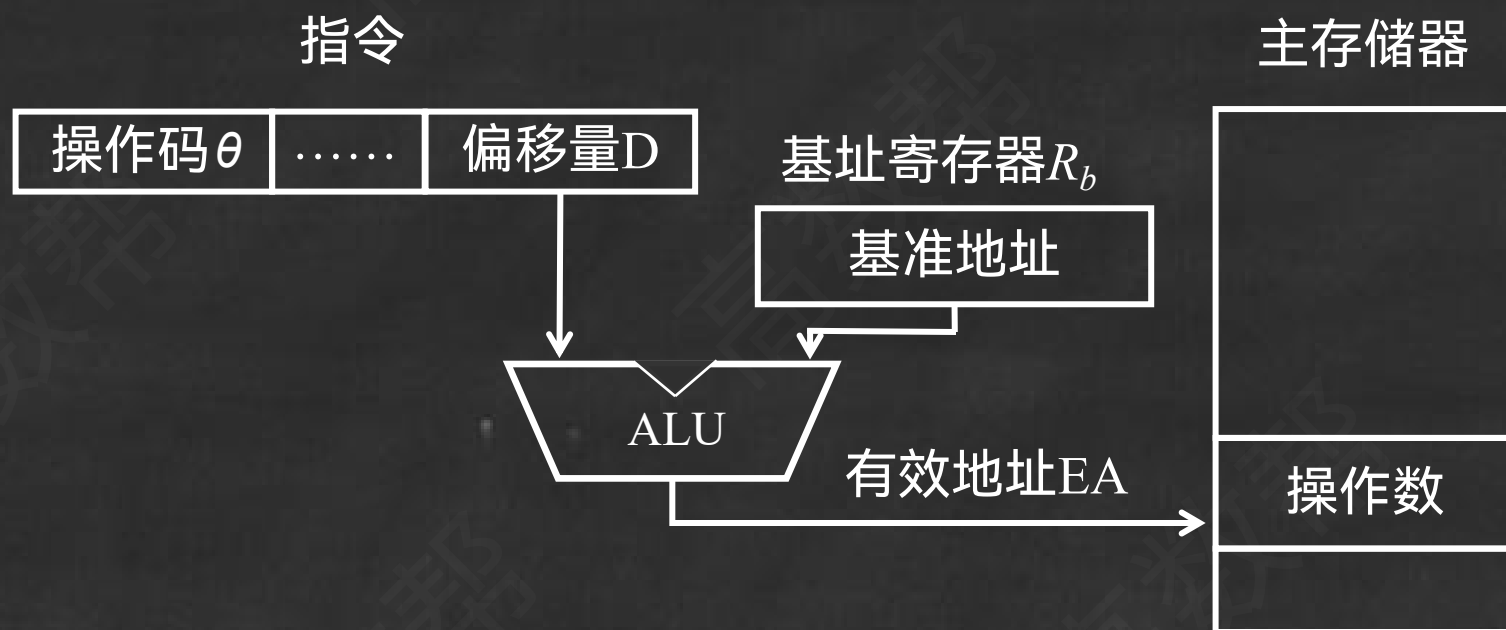
(1) 相对寻址 基准地址存放在程序计数器PC中，即 $EA = (PC) + D$



二、数据寻址：寻找操作数的地址

(2) 基址寻址

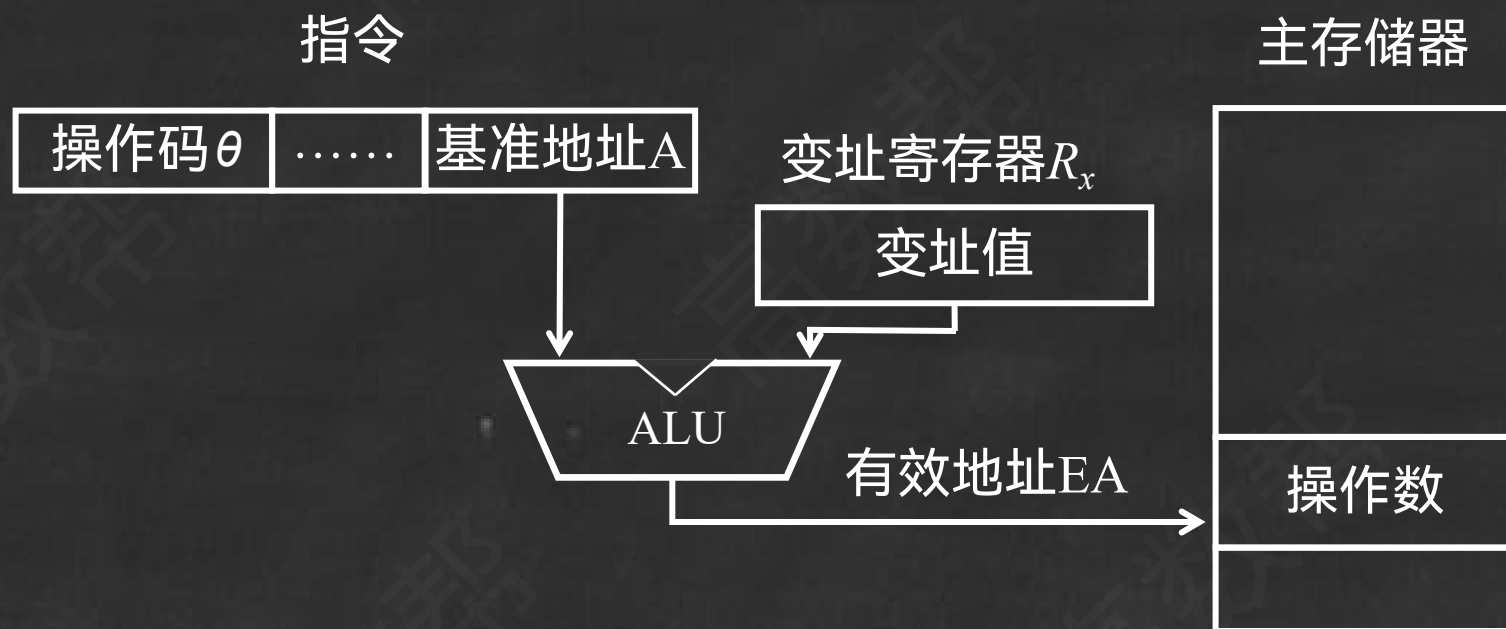
基准地址放在基址寄存器 R_b 中，即 $EA = (R_b) + D$



二、数据寻址：寻找操作数的地址

(3) 变址寻址

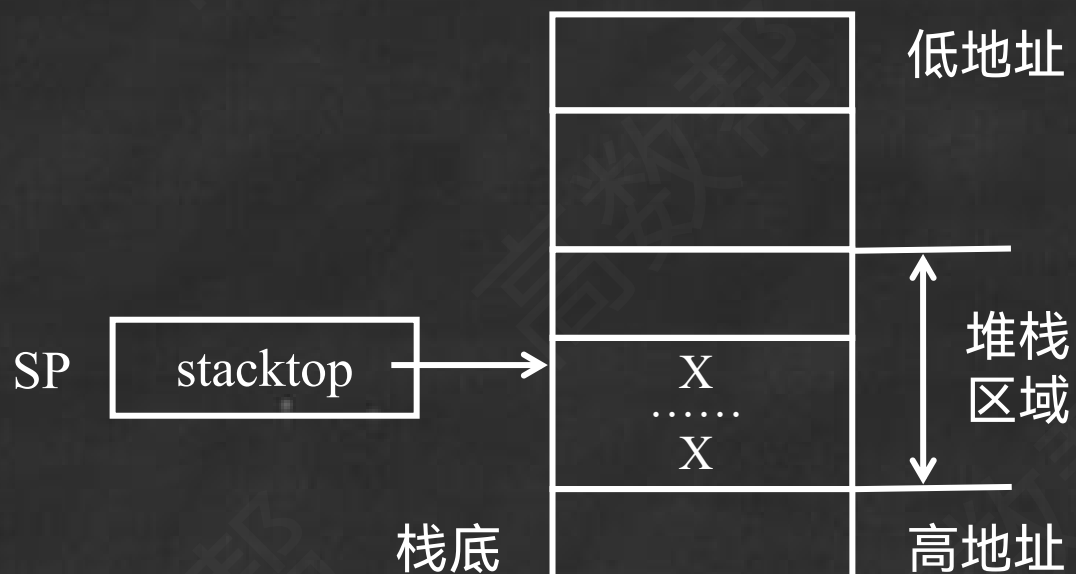
基准地址放在变址寄存器 R_x 中，即 $EA = (R_x) + D$



二、数据寻址：寻找操作数的地址

8、堆栈寻址

操作数地址由堆栈指针SP指定。



扫码观看
视频讲解更清晰

【题10】（根据有效地址求寻址方式题型）一种二地址RS型指令的结构如下：

6位	4位	1位	2位	16位
OP	通用寄存器	I	X	偏移量D

其中I为间接寻址标志位，X为寻址模式字段，D为偏移量字段。通过I，X，D的组合，可构成如下寻址方式；请写出 6种寻址方式的名称。

寻址方式	I	X	有效地址E算法	说明
(1)	0	00	$E = D$	
(2)	0	01	$E = (PC) \pm D$	PC为程序计数器
(3)	0	10	$E = (R_2) \pm D$	R_2 为变址寄存器
(4)	1	11	$E = (R_3)$	
(5)	1	00	$E = (D)$	
(6)	0	11	$E = (R_1) \pm D$	R_1 为基址寄存器

解：(1)直接寻址 (2)相对寻址 (3)变址寻址 (4)寄存器间接寻址 (5)间接寻址 (6)基址寻址

【题11】（求有效地址和操作数内容题型）一条双字长的取数指令(LDA)存于存储器的200和201单元，其中第一个字为操作码OP和寻址特征M，第二个字为形式地址A。假设PC当前值为200(还没有取该条双字长指令)，变址寄存器的内容为100，基址寄存器的内容为200，存储器相关单元的内容见表。存储器相关单元的内容：

地址	201	300	400	401	500	501	502	700
内容	300	400	700	501	600	700	900	401

下表的各列分别为寻址方式，该寻址方式下的有效地址以及**取数指令执行结束后累加器ACC的内容**(ACC中存放的其实就是有效地址对应的操作数)，试补全表。

寻址方式	有效地址EA	累加器ACC的内容
立即寻址	----	300
直接寻址	300	400
间接寻址	400	700
相对寻址	502	900
变址寻址		
基址寻址		

(1)直接寻址：由于直接寻址的有效地址EA为形式地址本身，因此直接寻址的有效地址为300，根据题目给出的表格可知，地址为300对应的内容为400。

(2)间接寻址：间接寻址中根据形式地址寻找到的内容才是真正的有效地址，即根据存储器的内容300找到的400才是间接寻址的有效地址，故有效地址为400，地址为400对应的内容为700。

(3)相对寻址：相对寻址中形式地址加上PC的内容为有效地址，PC当前值为200，当取出一条指令后，变为202，故有效地址为 $202+300=502$ ，地址为502对应的内容为900。

【题11】（求有效地址和操作数内容题型）一条双字长的取数指令(LDA)存于存储器的200和201单元，其中第一个字为操作码OP和寻址特征M，第二个字为形式地址A。假设PC当前值为200(还没有取该条双字长指令)，变址寄存器的内容为100，基址寄存器的内容为200，存储器相关单元的内容见表。存储器相关单元的内容：

地址	201	300	400	401	500	501	502	700
内容	300	400	700	501	600	700	900	401

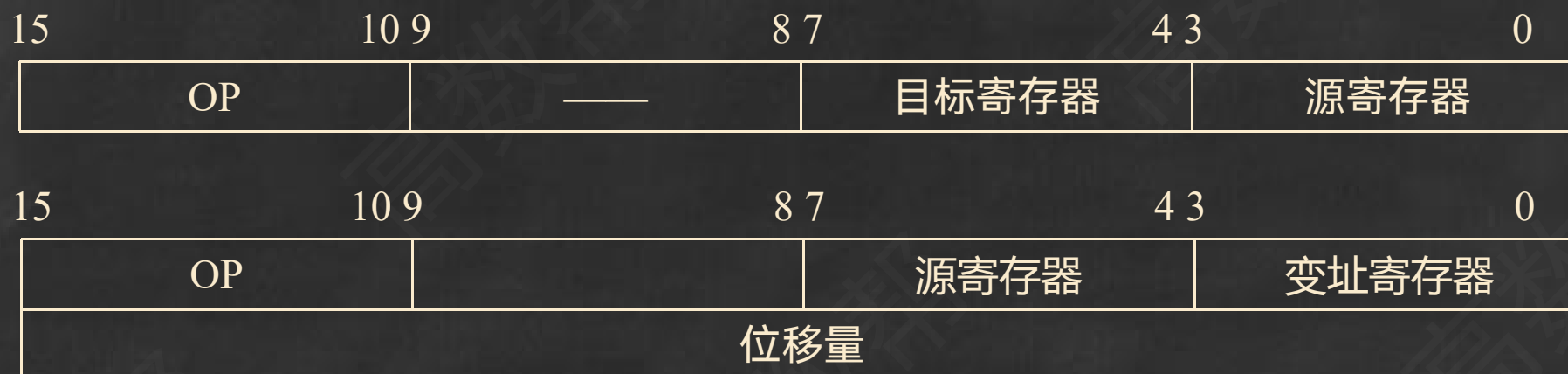
下表的各列分别为寻址方式，该寻址方式下的有效地址以及取数指令执行结束后累加器ACC的内容(ACC中存放的其实就是有效地址对应的操作数)，试补全表。

寻址方式	有效地址EA	累加器ACC的内容
立即寻址	----	300
直接寻址	300	400
间接寻址	400	700
相对寻址	502	900
变址寻址	400	700
基址寻址	500	600

(4)变址寻址：变址寻址的有效地址为形式地址加上变址寄存器的内容，因此有效地址为 $100+300=400$ ，地址为400对应的内容为700。

(5)基址寻址：基址寻址的有效地址为形式地址加上基址寄存器的内容，因此有效地址为 $200+300=500$ ，地址为500对应的内容为600。

【题12】（指令格式与寻址方式分析题型）某16位机器使用的指令结构如下所示，试分析以下2种指令格式及寻址方式特点。



解：1.第一种指令：

- ①单字长二地址指令 ②RR型指令，源寄存器和目标寄存器都是通用寄存器（可分别指定16个）
- ③操作码字段OP可以指定 $2^6=64$ 种操作 ④这种指令结构常用于算数逻辑运算类指令

2.第二种指令：

- ①双字长二地址指令，用于访问存储器。 ②操作码字段可以指定 $2^6=64$ 种操作。
- ③RS型指令，一个操作数在源寄存器中，另一个操作数在存储器中（有效地址可通过变址寻址求得，即有效地址等于变址寄存器（共16个）内容加上位移量）。

课时7 CPU的结构和功能

考 点	重要程度	分值	常考题型
1. CPU的基本功能与组成	★★★	2 ~ 4	填空、选择、判断
2. 指令周期	★★★	2 ~ 4	填空、选择、判断
3. 指令流水线	★★★★	2 ~ 8	填空、计算、分析
4. 中断系统	★★★	2 ~ 4	选择、简答



考点1 CPU的基本功能与组成 了解



一、CPU基本功能

取指令 $\xrightarrow{\text{操作控制、时间控制}}$ 执行指令

中央处理器

程序控制功能 { 指令控制 (程序的顺序控制)
操作控制 (将一条指令的若干操作信号送往相应部件)

操作定序功能 时间控制 (指令各个操作实施时间的定时)

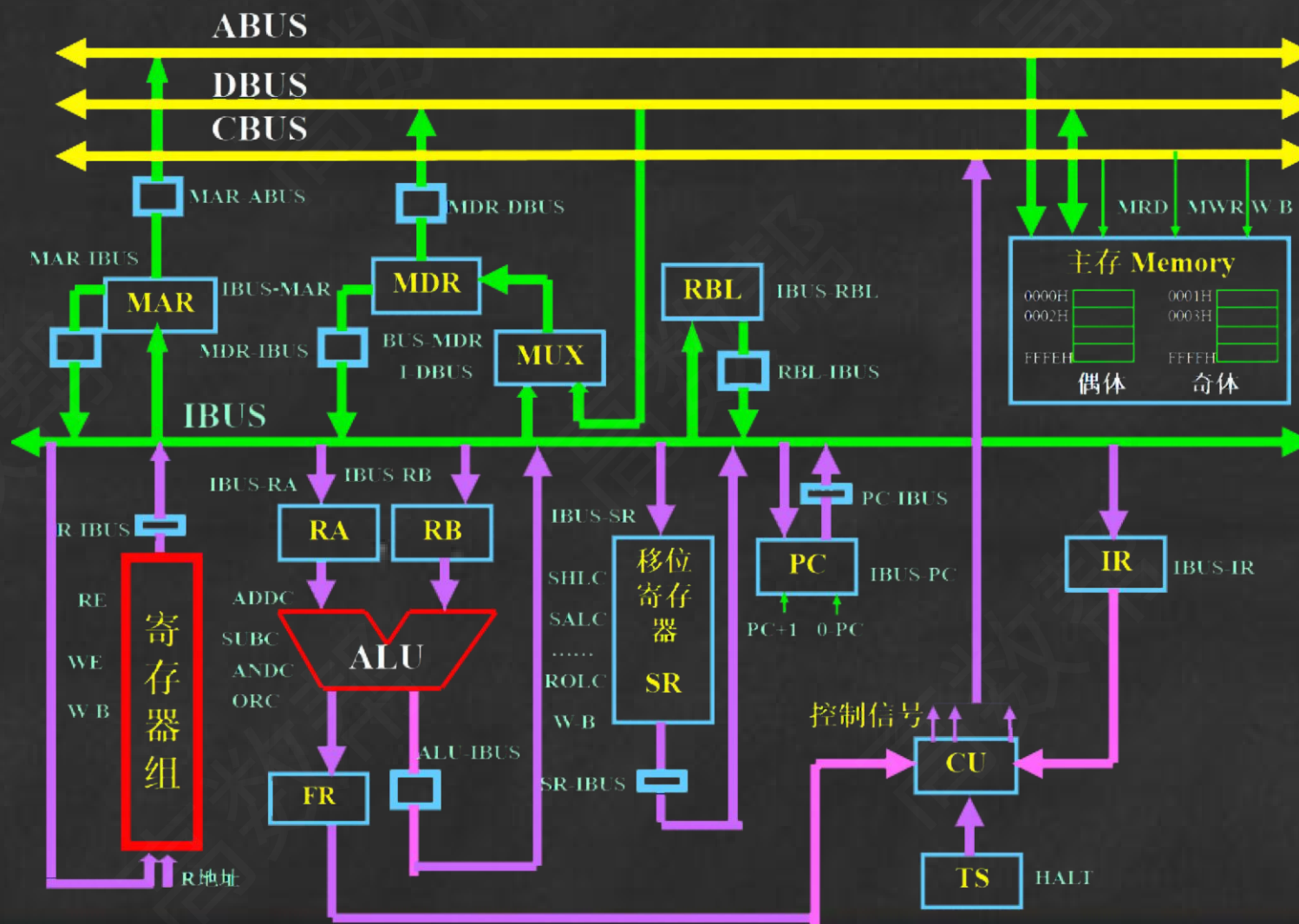
数据处理功能 数据加工 (CPU的根本任务: 算术运算和逻辑运算)

中断处理功能



二、CPU基本组成

CPU = 运算器 + 控制器，通过CPU内部数据通路连接如下图所示：



二、CPU基本组成

1、**算数逻辑运算单元ALU**：执行算数运算、逻辑运算与逻辑测试

2、**操作控制单元CU**：

根据指令操作码和时序信号产生各种操作信号，从而正确的选择数据通路把数据打入某个寄存器，实现取指和执行指令。

根据设计方法不同，可分为 { 硬布线控制器：采用时序逻辑技术实现
微程序控制器：采用存储逻辑技术实现

3、**时序产生器TS**：提供定时和时序信号，从而对操作信号实施时间上的控制。

二、CPU基本组成



4、CPU的寄存器

(1) 程序计数器PC(Programming Counter)

- ① 用来存放正在执行的指令的地址或下一条指令的地址。
- ② 顺序执行时，每执行一条指令，PC的值应加1。
- ③ 要改变程序执行顺序的情况时，一般由转移类指令将转移目标地址送往PC，可实现程序的转移。



扫码观看
视频讲解更清晰

二、CPU基本组成

4、CPU的寄存器

(2) 指令寄存器IR(Instruction Register)

- ① 指令寄存器用来存放从指令存储器中取出的待执行的指令。
- ② 在执行该指令的过程中，指令寄存器的内容不允许发生变化，以保证实现指令的全部功能。

二、CPU基本组成

4、CPU的寄存器

(3) 状态字寄存器(PSW)或称标志寄存器 (FR)

用于存放状态标志，可用做程序分支的依据

	N	Z	V	P	C
0	非负	非零	无溢出	奇数个1	无进位
1	负数	零	溢出	偶数个1	有进位

(4) 主存地址寄存器MAR: 存放地址

(5) 主存数据寄存器MDR: 存放数据

二、CPU基本组成

4、CPU的寄存器

(3) 状态字寄存器(PSW)或称标志寄存器 (FR)

用于存放状态标志，可用做程序分支的依据

	N	Z	V	P	C
0	非负	非零	无溢出	奇数个1	无进位
1	负数	零	溢出	偶数个1	有进位

(4) 主存地址寄存器MAR: 存放地址

(5) 主存数据寄存器MDR: 存放数据

二、CPU基本组成

4、CPU的寄存器

(6) 寄存器组

① 通用寄存器:

AX-累加器, BX-基址寄存器, CX-计数寄存器, DX-数据寄存器,
SP-堆栈指针, BP-基址指针, SI-源变址寄存器, DI-目的变址寄存器。

② 段寄存器:

CS-代码段寄存器, DS-数据段寄存器, SS-堆栈段寄存器, ES-附加段寄存器。

【题1】 保存CPU将要访问的程序中指令的主存地址的寄存器（或描述为：跟踪指令后继地址的寄存器）是 程序计数器PC。

【题2】 保存正在执行指令的寄存器是 指令寄存器IR。



扫码观看
视频讲解更清晰

考点2 指令周期 了解



一、指令周期基本概念

指令周期：从主存中取出一条指令，并执行完该指令的时间。

CPU周期（又称机器周期、工作周期）：

把指令周期分成若干个子周期，每个子周期完成不同工作，称为CPU周期。

如取指周期、执行周期。不同的指令，可能包含不同数目的CPU周期。

时钟周期（又称节拍、T周期）：

把机器周期分成若干时间段，每个时间段对应一个电平信号，完成一次数据通路中的基本操作，称为时钟周期或节拍，是处理操作的最基本单位。

考点2 指令周期 了解



一、指令周期基本概念

指令周期：从主存中取出一条指令，并执行完该指令的时间。

CPU周期（又称机器周期、工作周期）：

把指令周期分成若干个子周期，每个子周期完成不同工作，称为CPU周期。

如取指周期、执行周期。不同的指令，可能包含不同数目的CPU周期。

时钟周期（又称节拍、T周期）：

把机器周期分成若干时间段，每个时间段对应一个电平信号，完成一次数据通路中的基本操作，称为时钟周期或节拍，是处理操作的最基本单位。

一、指令周期基本概念

时钟脉冲信号（CLK，时钟信号，定时脉冲）： 时序系统控制机器运行的基本定时信号。



【题3】 指令周期指 (C)

- A、CPU从主存中取出一条指令的时间
- B、CPU执行一条指令的时间
- C、CPU从主存取出一条指令加上执行这条指令的时间
- D、时钟周期时间

【题4】 处理操作的最基本单位是 (C)

- A、指令周期
- B、机器周期
- C、时钟周期
- D、时钟脉冲

考点3 指令流水

理解



一、基本概念

- 1、**并发**：两个或两个以上事件在 同一时间段 发生
- 2、**并行**：两个或两个以上事件在 同一时刻 发生

指令并行技术三种形式		
时间并行（时间重叠）	空间并行（资源重复）	时间+空间并行
让一条指令的多个处理过程在时间上相互错开，轮流使用同一套硬件设备的各个部件，以加快硬件周转而赢得速度。实现方式就是采用流水处理部件。	以资源数量取胜，如多核系统。	如超标量流水线技术

【题5】 假设将指令分取指令IF、指令译码ID、执行运算EX、结果写回WB共4个处理过程，共有2条指令连续输入。

(1) 画出指令周期流程图

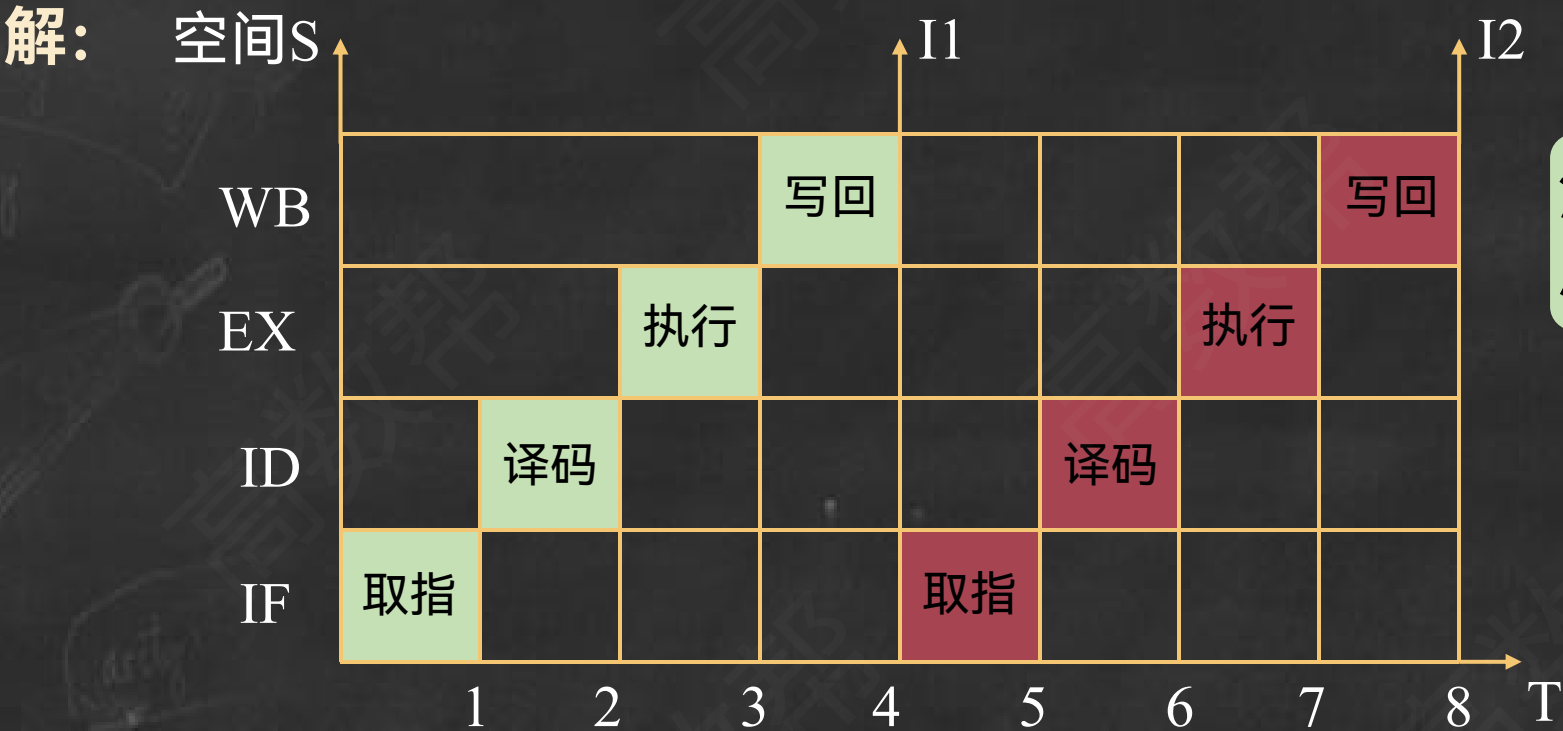
解：



扫码观看
视频讲解更清晰

【题5】 假设将指令分取指令IF、指令译码ID、执行运算EX、结果写回WB共4个处理过程，共有2条指令连续输入。

(2) 画出非流水线时空图



注意:m个子过程, n条指令
周期数: $m \times n$ 个

$4 \times 2 = 8$

非流水线时空图

【题5】 假设将指令分取指令IF、指令译码ID、执行运算EX、结果写回WB共4个处理过程，共有2条指令连续输入。

(3) 假设存在一条指令流水线，画出标量流水线时空图



$4+(2-1)=5$

注意:m个子过程, n条指令
周期数: $m+n-1$ 个

标准流水线时空图

【题6】 设指令由取指、分析、执行3个子部件完成，每个子部件的工作周期均为 Δt ，采用常规标量流水线处理器。若连续执行10条指令，则需要的时间为（ C ）。

A. $8\Delta t$

B. $10\Delta t$

C. $12\Delta t$

D. $14\Delta t$

解：

只有第一条指令需要3个工作周期，其余9条指令每隔1个工作周期即可得到结果，

所以需要 $(3 + 9) \times \Delta t = 12\Delta t$ 。

因此完成 n 条指令在 m 段流水线上共需 $(m+n-1) \times \Delta t$ 的时间。

二、影响流水线性能主要问题

1、资源相关

(1) 概念：多条指令进入流水线后在同一时钟周期内争用同一功能部件所发生的冲突。

(2) 解决办法：后边指令拖一拍再推进；增设一个功能部件

指令 \ 时钟	1	2	3	4	5	6	7	8
I1 (Load)	IF	ID	EX	MEM	WB			
I2		IF	ID	EX	MEM	WB		
I3			IF	ID	EX	MEM	WB	
I4				IF	ID	EX	MEM	WB
I5					IF	ID	EX	MEM

二、影响流水线性能主要问题

2、数据相关:

- ① **RAW(Read After Write):** 写后读, 后面指令用到前面指令所写的数据
- ② **WAW(Write After Write):** 写后写, 两条指令写同一个单元。在简单流水线中没有此类相关, 因为不会乱序执行。
- ③ **WAR(Write After Read):** 读后写, 后面指令覆盖前面指令所读的单元, 在简单流水线中没有此类相关。

解决办法: 可以推后后继指令对相关单元的读操作, 或设置相关的直接通路。

【题7】 以下指令发生了 写后读RAW 数据相关的冲突。

ADD R1,R2,R3 $R2+R3 \rightarrow R1$

SUB R4,R1,R5 $R1-R5 \rightarrow R4$

AND R6,R1,R7 $R1 \wedge R7 \rightarrow R6$

指令\时钟	1	2	3	4	5	6	7	8
ADD	IF	ID	EX	MEM	WB			
SUB		IF	ID	EX	MEM	WB		
AND			IF	ID	EX	MEM	WB	

【题8】 流水线中有三类数据相关冲突：写后读（RAW）相关；读后写（WAR）相关；写后写（WAW）相关。判断以下三组指令各存在哪种冲突类型的数据相关，并分析该冲突对结果的影响。

(1) I1 ADD R1, R2, R3 ; $(R2) + (R3) \rightarrow R1$
 I2 SUB R4, R1, R5 ; $(R1) - (R5) \rightarrow R4$

解：第（1）组指令发生写后读RAW相关。

I1指令运算结果应先写入R1，然后在I2指令中读出R1内容。

由于I2指令进入流水线，变成I2指令在I1指令写入R1前就读出R1内容。

【题8】 流水线中有三类数据相关冲突：写后读（RAW）相关；读后写（WAR）相关；写后写（WAW）相关。判断以下三组指令各存在哪种冲突类型的数据相关，并分析该冲突对结果的影响。

(2) I3 STO M(x), R3 ; (R3) $\rightarrow$ M(x), M(x)是存储器单元
I4 ADD R3, R4, R5 ; (R4) + (R5) $\rightarrow$ R3

解：第（2）组指令发生读后写WAR相关。

I3指令应先读出R3内容并存入存储单元M(x)，然后在I4指令中将运算结果写入R3。但由于I4指令进入流水线，变成I4指令在I3指令读出R3内容前就写入R3。

【题8】 流水线中有三类数据相关冲突：写后读（RAW）相关；读后写（WAR）相关；写后写（WAW）相关。判断以下三组指令各存在哪种冲突类型的数据相关，并分析该冲突对结果的影响。

(3) I5 MUL R3, R1, R2 ; $(R1) \times (R2) \rightarrow R3$
 I6 ADD R3, R4, R5 ; $(R4) + (R5) \rightarrow R3$

解：第（3）组指令发生写后写WAW相关。

如果I6指令的加法运算完成时间早于I5指令的乘法运算时间，变成指令I6在指令I5写入R3前就写入R3，导致R3的内容错误。



扫码观看
视频讲解更清晰

【题9】 指令序列如下， 其在指令流水线中的执行过程见表： 表 指定序列的执行过程

I1 LOAD R1, [a]
I2 LOAD R2, [b]
I3 ADD R1, R2
I4 STORE R2, [x]

指令	时间单元													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1	IF	ID	EX	M	WB									
I2		IF	ID	EX	M	WB								
I3			IF				ID	EX	M	WB				
I4							IF				ID	EX	M	WB

请分析这4条指令执行过程中， I3的ID段， 和I4的IF和ID段被阻塞的原因各是什么？

解： 各条指令语句的含义如下：

I1 LOAD R1, [a] // [a]->R1
I2 LOAD R2, [b] // [b]->R2
I3 ADD R1, R2 // R1+R2->R2
I4 STORE R2, [x] // R2->[x]

(1) 指令I3需要指令I1与指令I2的数据， 所以存在写后读的数据相关， 指令I3推后了对R1和R2的读操作， 等到指令I1、 I2的结果分别写回寄存器R1和R2， 才能执行读寄存器操作， 所以I3的ID段被阻塞。

【题9】 指令序列如下， 其在指令流水线中的执行过程见表： 表 指定序列的执行过程

I1 LOAD R1, [a]
I2 LOAD R2, [b]
I3 ADD R1, R2
I4 STORE R2, [x]

指令	时间单元													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
I1	IF	ID	EX	M	WB									
I2		IF	ID	EX	M	WB								
I3			IF				ID	EX	M	WB				
I4							IF				ID	EX	M	WB

请分析这4条指令执行过程中， I3的ID段， 和I4的IF和ID段被阻塞的原因各是什么？

解： 各条指令语句的含义如下：

I1 LOAD R1, [a] // [a]->R1
I2 LOAD R2, [b] // [b]->R2
I3 ADD R1, R2 // R1+R2->R2
I4 STORE R2, [x] // R2->[x]

(2) 因为I3的ID段被阻塞， 所以I4的IF段被阻塞。另外， 由于指令I4和指令I3发生写后读的数据相关， 导致指令I4需要等I3将R2结果写回后才能读出R2， 因此指令14的ID段被阻塞。

二、影响流水线性能主要问题

3、控制相关:

(1) 概念: 由转移指令引起, 指令转到新的目标地址取指, 从而使流水线发生断流

二、影响流水线性能主要问题

3、控制相关:

(2) 解决办法:

- ① 延迟转移法: 先不排空流水线, 而让紧跟在转移指令之后已经进入流水线的少数几条指令继续完成。称为“先执行再转移”。
- ② 转移预测法: 依据指令过去的行为预测将来的行为, 将转移预测提前到取指阶段进行。采用硬件方法实现两路指令预取队列器以及目标指令cache。

三、流水线的性能

1、加速比 S_p

m 段的 **流水线的速度** 与等功能的 **非流水线的速度** 之比

设 m 段的流水线各段时间为 Δt

完成 n 条指令在 m 段流水线上共需 $T = (m + (n - 1)) \cdot \Delta t$

完成 n 条指令在等效的非流水线上共需 $T' = nm \cdot \Delta t$

则加速比
$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n - 1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$

三、流水线的性能

2、**吞吐率**：单位时间内流水线所完成指令或输出结果的数量

(1) **最大吞吐率**： $T_{pmax} = \frac{1}{\Delta t}$

(2) **实际吞吐率**：连续处理 n 条指令的吞吐率为 $T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$

【题10】指令流水线有取指令IF、指令译码ID、执行运算EX、访存MEM、结果写回WR共5个处理过程，共有20条指令连续输入此流水线。假设时钟周期为100ns。

(1) 求流水线的实际吞吐率（单位时间里执行完的指令数）

(2) 求流水线的加速比。

解： (1) $T = \frac{n}{(m + n - 1) \cdot \Delta t}$

$$T = \frac{20}{(5 + 20 - 1) \times 100 \times 10^{-9}} = 8.33 \times 10^6 \text{ 条 / 秒}$$

$$(2) S = \frac{nm \cdot \Delta t}{(m + n - 1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$

$$S = \frac{20 \times 5}{20 + 5 - 1} = 4.17$$



扫码观看
视频讲解更清晰

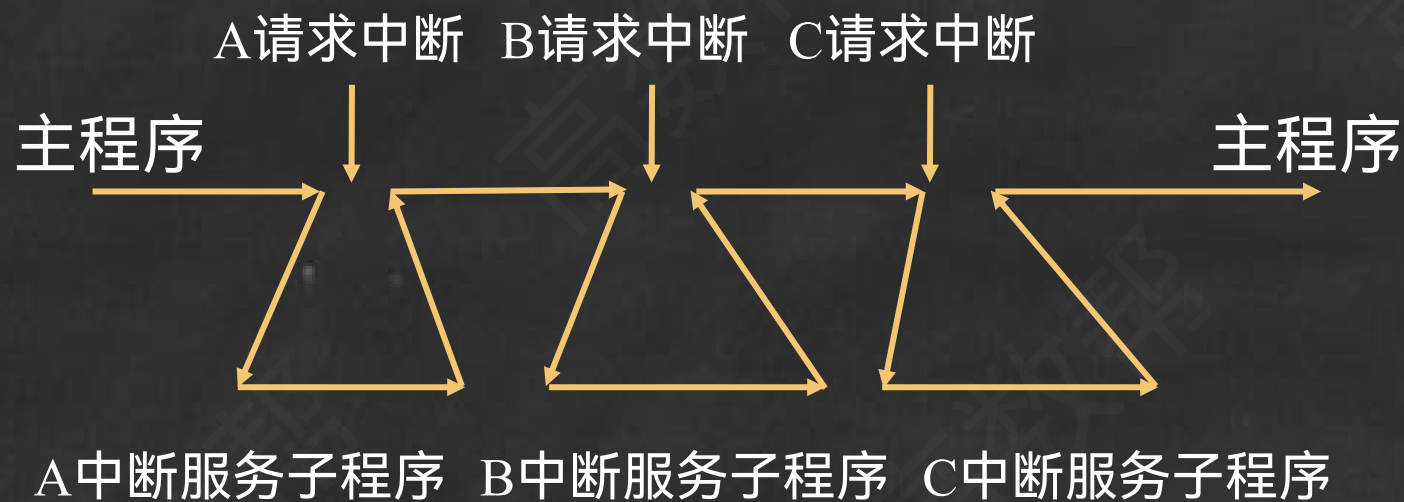
考点4 中断系统 理解



一、中断基本概念

1、中断：是指CPU暂时中止现行程序，转去处理随机发生的紧急事件，处理完后自动返回原程序的功能和技术。

2、中断系统：是计算机实现中断功能的软、硬件总称。



3、中断源：引起中断的各种事件。如程序错误、硬件故障、响应I/O设备。

4、中断基本类型:

- 内中断:** CPU内部硬件或软件引起的中断, 如单步中断、溢出中断。
- 外中断:** CPU以外的部件引起的中断。

根据该中断请求是否可在CPU内部屏蔽掉, 分为

- 可屏蔽中断
- 不可屏蔽中断

- 单级中断:** CPU执行中断服务程序过程不能被打断。
- 多级中断:** CPU执行中断服务程序过程, 可去响应级别更高的中断请求。



(a) 多级中断示意图

二、中断处理过程

中断处理过程分为5个阶段：中断请求、中断判优、中断响应、中断处理、中断返回。

1、中断请求：

中断请求触发器 (INTR)：记录中断源的触发器，一个中断源对应一个INTR，置1代表提出中断请求。可分散在各个中断源中，也可集中到CPU的中断系统内。

2、中断判优：

(1) **软件判优**：用程序检测优先级大小。

(2) **硬件判优**：根据不同的优先排队电路判优，速度快，成本高。

二、中断处理过程

3、中断响应:

(1) 中断响应条件:

- ① CPU收到中断请求信号
- ② CPU开中断, 即中断允许触发器 $EINT=1$

(2) 中断响应时间: 一条指令执行完毕, CPU发出查询中断信号

二、中断处理过程

4、中断处理：

- (1) 保护现场：**保护程序断点以及通用寄存器、状态寄存器。过程需要关中断，防止被其他中断打断。断点可以存入堆栈。
- (2) 中断服务：**寻找中断服务程序入口地址（向量地址），并视不同中断源执行不同中断服务程序。
- (3) 恢复现场：**退出服务程序前，将原程序中断时的现场恢复到原来寄存器中。

5、中断返回：

执行中断返回指令，堆栈弹出断点地址给PC，CPU转移到原来程序断点继续运行。



三、多级中断的中断屏蔽技术：

- 1.当系统处于开中断状态，多级中断可实现优先级别高的中断源有权中断优先级别低的中断源，保护多个断点可通过堆栈先进后出的特点实现。
- 2.多级中断利用中断屏蔽技术，设定中断接口或者CPU内部屏蔽字，选择允许某些中断源的请求而屏蔽某些中断源，实现中断源管理和设置中断优先级，构成想要的中断嵌套。

【题11】 中断服务程序的最后一条指令是（ C ）。

A.转移指令

B.出栈指令

C.中断返回指令

D.开中断指令



扫码观看
视频讲解更清晰

【题12】 中断处理过程可以嵌套进行，优先级高的设备可以中断优先级低的中断服务程序。

【题13】 在什么条件和什么时间，CPU可以响应I/O的中断请求？

答：

CPU响应I/O中断请求的条件和时间是：

当中断允许状态为1($EINT=1$)，且至少有一个中断请求被查到，则在一条指令执行完时，响应中断。

【题14】 中断向量可提供 (C)

A、被选中的设备的地址

B、传送数据的起始地址

C、中断服务程序的入口地址

D、主程序的断点地址

【题15】 为了便于实现多级中断，保存现场信息最有效的方法是采用 堆栈。

课时8 CPU的控制单元

考 点	重要程度	分值	常考题型
1.控制单元的功能	★★★	2 ~ 4	选择、填空、判断
2.微程序控制器	★★★	2 ~ 4	选择、填空、简答



考点1 控制单元功能 了解



一、基本概念

1、操作控制器：根据指令操作码和时序信号产生各种操作信号，从而正确的选择数据通路把数据打入某个寄存器，实现取指和执行指令。

根据设计方法不同，可分为时序逻辑型和存储逻辑型：

- ①组合逻辑控制器/硬布线控制器：采用时序逻辑技术实现
- ②微程序控制器：采用存储逻辑技术实现



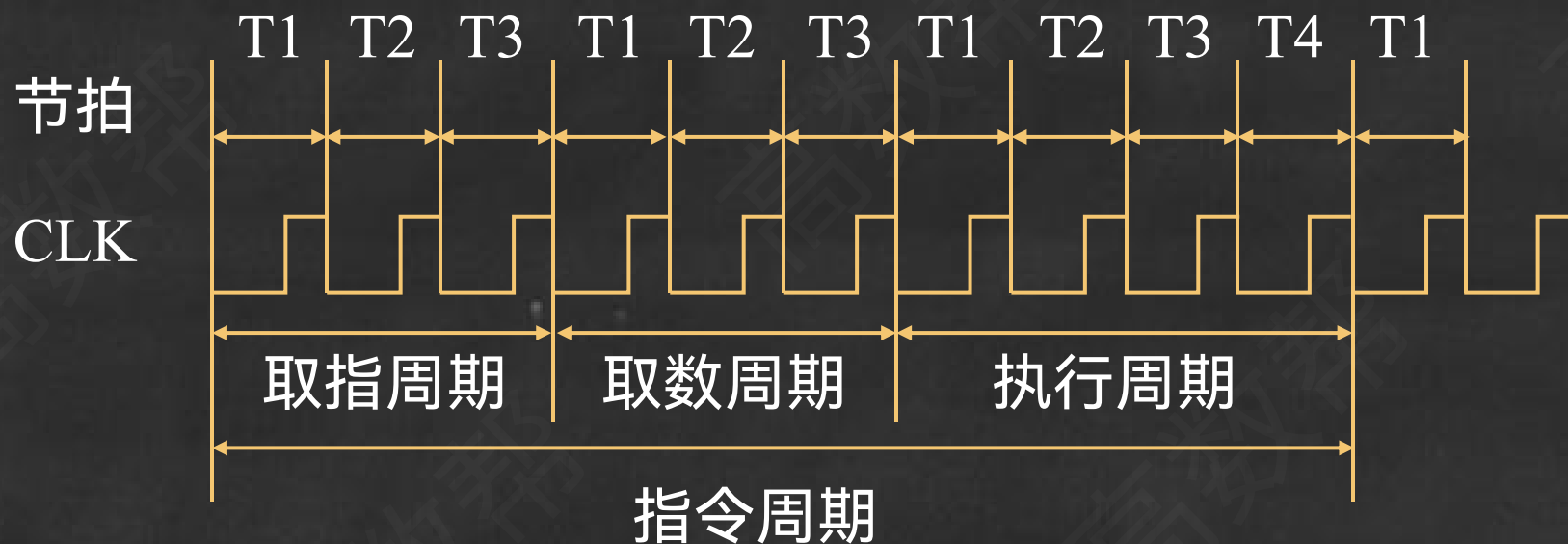
扫码观看
视频讲解更清晰

一、基本概念

2、时序产生器：提供定时和时序信号，从而对操作信号实施时间上的控制。

①组合逻辑控制器：机器周期、时钟周期及时钟脉冲三级时序。

②微程序控制器：时钟周期和时钟脉冲二级时序。



【题1】 (A) 不是常用三级时序系统中的一级。

A.指令周期

B.机器周期

C.节拍

D.定时脉冲

二、控制方式

1、**同步控制方式**：指令的机器周期数和时钟周期数不变

2、**异步控制方式**：每条指令需要多长时间就占多长时间

3、**联合控制方式**

①大部分指令在固定的周期内完成，少数难以确定的操作采用异步方式

②机器周期的节拍脉冲固定，但是各指令的机器周期数不固定（微程序控制器采用）

三、硬布线控制器/组合逻辑控制器

1、实现方法

通过逻辑电路直接连线而产生的，把控制部件看做产生专门固定时序控制信号的逻辑电路

2、设计目标

- ①使用最少元件
- ②速度最高

考点2 微程序控制器 理解



一、基本概念

1、微程序控制器基本思想：

仿照解题的方法，把一条指令编制成微指令，存放到控制存储器里。
运行时，从控存中取出微指令，产生指令运行所需的操作控制信号。
微程序设计技术是用软件方法来设计操作控制器的技术。



扫码观看
视频讲解更清晰

一、基本概念



2、微命令与微操作

- ①微命令：控制部件向执行部件发出的各种控制命令叫做微命令，它是构成控制序列的最小单位。
- ②微操作：微命令的操作过程，它是执行部件中最基本的操作。
- ③微命令与微操作关系：微命令和微操作是一一对应的。微命令是微操作的控制信号，微操作是微命令的操作过程。

微操作分类：

- ①互斥的微操作：不能同时或不能在同一个节拍内并行执行的微操作，可以组合编码。如ALU的+，-，移位等微操作互斥。
- ②相容的微操作：能够同时或在同一个节拍内并行执行的微操作，必须各占一位。

【题2】 下列说法正确的是 (AD)

A、同一个CPU周期中，可以并行执行的微操作称为相容性微操作

B、同一个CPU周期中，不可以并行执行的微操作称为相容性微操作

C、同一个CPU周期中，可以并行执行的微操作称为相斥性微操作

D、同一个CPU周期中，不可以并行执行的微操作称为相斥性微操作

一、基本概念



3、微指令与微周期

(1) 微指令：是微命令的组合，把在同一CPU周期内并行执行的微操作控制信息，存储在控制存储器里。

组成：

- ①微命令字段 (微操作控制字段)：用以产生某一步操作所需的各个微操作控制信号。
- ②顺序控制字段(微地址字段)：产生下一条将要执行的微指令地址

(2) 微周期：从CM中读取一条微指令并执行相应的微操作所需的时间

一、基本概念



4、工作程序与微程序

(1) 微程序：一系列微指令的有序集合就是微程序，以微指令为单位存放在控制存储器中。

一段微程序对应一条机器指令。

(2) 工作程序：一系列机器指令的有序集合就是程序，以机器指令为单位存放在主存中。



扫码观看
视频讲解更清晰

一、基本概念

5、主存储器与控制存储器

(1) **主存**：指令、程序、地址与主存（主存储器）有关

(2) **控存**：微指令、微程序、微地址与控存（控制存储器）有关

二、设计微指令的目标

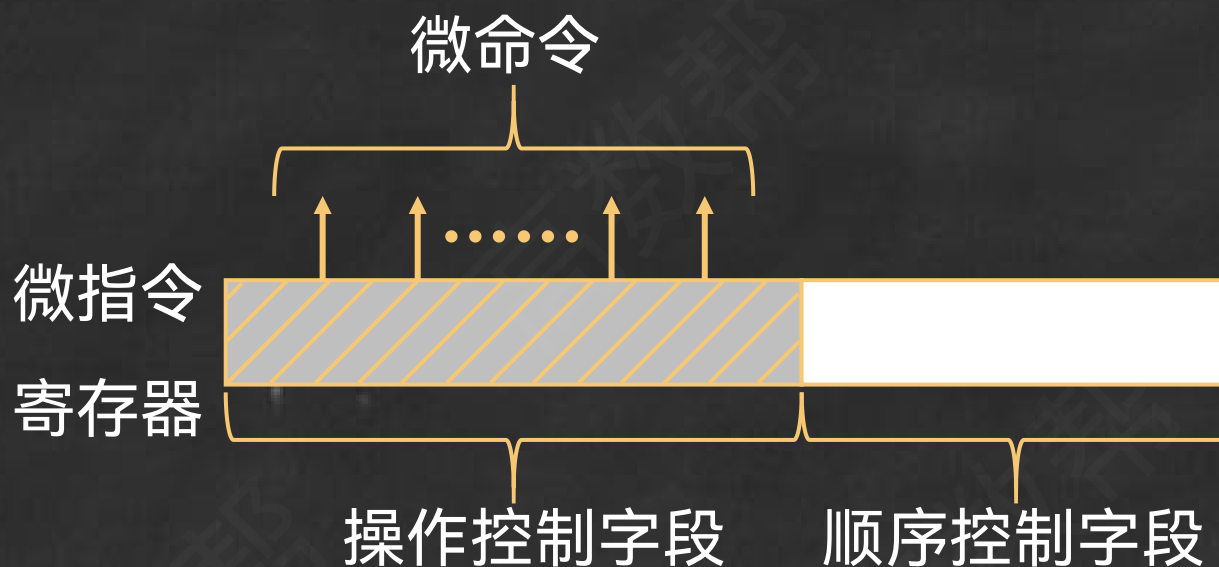
- ① 有利于缩短微指令的长度
- ② 有利于缩小CM的容量
- ③ 有利于提高微程序的执行速度
- ④ 有利于对微指令的修改
- ⑤ 有利于提高微程序设计的灵活性



三、微命令编码方法

1、**直接控制编码（不译码法）**：每一位代表一个微命令，1有效，不需要进行译码。

优点：控制—简单、速度—快、并行性—好、长度—长。



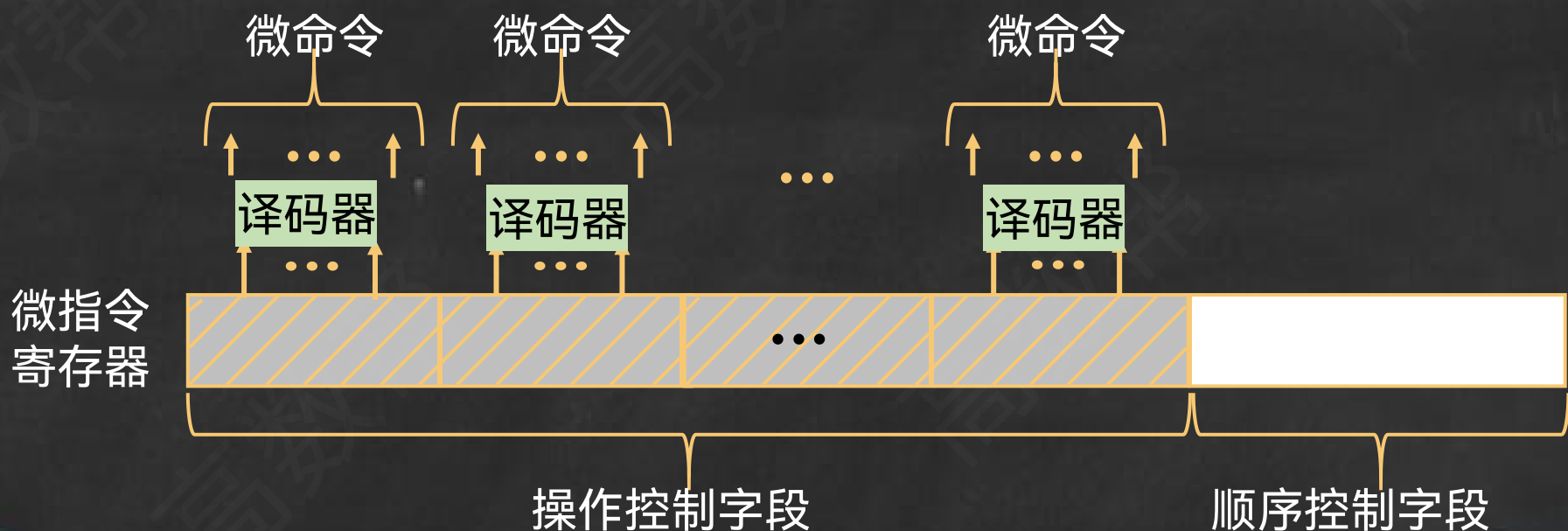
三、微命令编码方法

2、字段直接编码法：把相斥性微命令组合在同一字段中，采用编码方式存取。

把相容性微命令组合在不同字段中，各段单独译码。

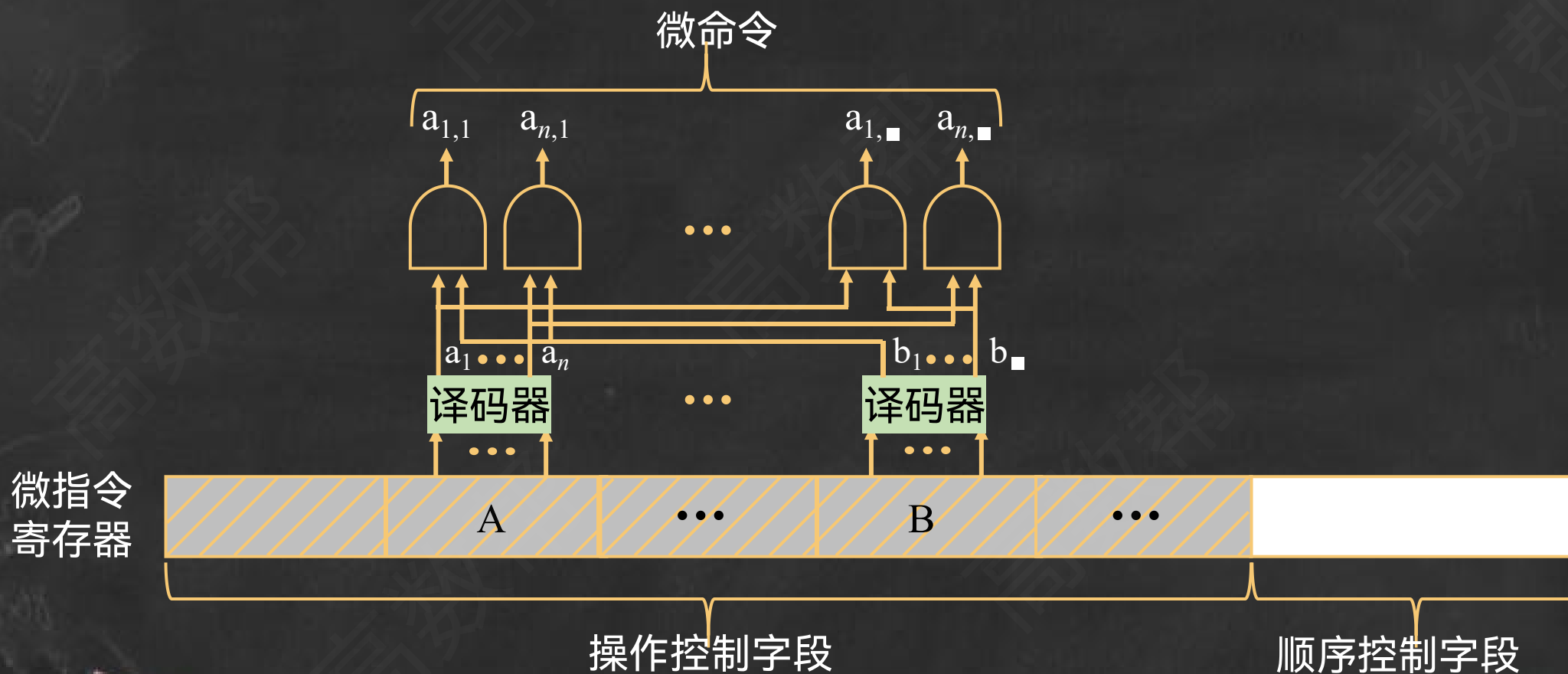
优点：可以避免互斥，使指令字大大缩短

缺点：增加了译码电路，使微程序的执行速度减慢



三、微命令编码方法

- 3、**字段间接编译法**：将前两种结合在一起，兼顾两者特点。字段的某些编码不能独立地定义某些微命令，而需要与其他字段的编码来联合定义。



四、微地址的形成方式

1、计数器增量方式: μPC

微命令字段	转移微地址	转移方式
	————— 顺序控制字段 —————	

- (1) **顺序执行**: 转移方式 = J_0 , $\mu PC+1 \Rightarrow \mu PC$, 由 μPC 提供后继微地址
- (2) **无条件转移**: 转移方式 = J_1 , 由转移微地址字段给出转移的微地址
- (3) **条件转移**: 转移方式 = J_2 , 条件成立, 则由转移微地址字段给出转移的微地址

优点: 简单、易于掌握, 编制微程序容易

缺点: 这种方式不能实现两路以上的并行微程序转移, 因而不利于提高微程序的执行速度。

四、微地址的形成方式

2、断定测试转移方式

微命令字段	测试字段	下地址字段
	————— 顺序控制字段 —————	

微程序顺序执行时： 由下地址字段直接提供后继微地址

微程序有分支时： 由测试字段给出应测试的状态，根据测试的状态标志位产生相应的后继微地址

五、微指令格式

1、水平型微指令：

一条微指令中并行定义多个微命令，并且在数据通路中能够并行执行多种微操作的微指令格式。一般采用直接控制编码、字段直接编码及字段间接编码。

优点：并行性高

缺点：微指令结构较长



扫码观看
视频讲解更清晰

五、微指令格式

2、垂直型微指令：

一次只能执行一到二个微命令的微指令。设置微操作码字段，由微操作码规定微指令功能，类似于机器指令结构。

优点：微指令结构较短

缺点：编写的微程序较长